

G H Raison College of Engineering & Management

An Empowered Autonomous Institute, Affiliated to Savitribai Phule Pune University

Approved by AICTE, New Delhi and Recognized by Govt. of Maharashtra, NAAC Accredited "A+" Grade

Navin Gat No.1200, Domkhel Road, Wagholi, Pune - 412 207 (India)

T: +91 - 9604787185/186 | **E:** ghrcem.pune@raisoni.net | **W:** www.ghrcemp.raisoni.net

Department of Information Technology

SY. B.Tech 2023 Scheme (As per NEP 2020)

Semester- III

UNIT 2.

LINKED LIST, SORTING AND SEARCHING TECHNIQUES

Mapping of Course Outcomes- CO2

CREATE LINKED LIST, STACK, QUEUE AND USE HASHING & COLLISION RESOLUTION
TECHNIQUES TO SOLVE COMPLEX PROBLEMS FROM DIFFERENT DOMAINS

Prepared by

Prof. Yogesh Gite

Topics

3

- Sorting Techniques - bubble sort, selection sort, insertion sort & their Analysis.
- Linked List:
 - Introduction, Representation of Linked List, Memory Allocation, Linked List v/s Array,
 - Types of Linked List - Singly Linked List, Circularly Linked Lists, Two-Way Lists (or Doubly Linked Lists),
 - Operations on Singly Linked List and Doubly Linked List: - Insertion, Deletion, Traversing.
 - Advantages, Disadvantages, Applications of Arrays and Linked List.
 - Singly Linked List Application-Polynomial Representation and Addition.
- Searching Techniques:
 - Linear Search,
 - Binary Search,
 - Hashing, Hash Functions, properties of good hash function, types of Hash Functions, Collision resolution Techniques.

Linked List

- ❑ **Linear**: Components are **ordered** in a line (hence called Linear linked lists).
- ❑ A **list** is a **linear** collection of **varying** length of **homogeneous** components.
- ❑ **Homogeneous**: All components are of the same type.

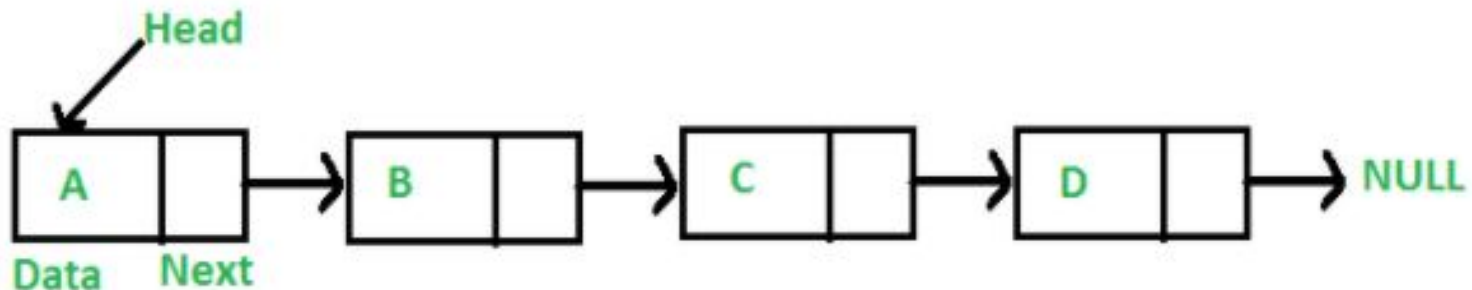


Linked List

It is a **linear data structure**, in which elements are **not stored at contiguous memory locations**.

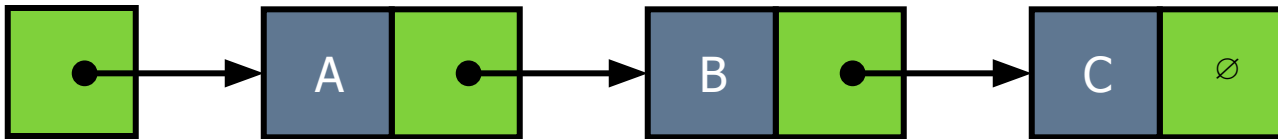
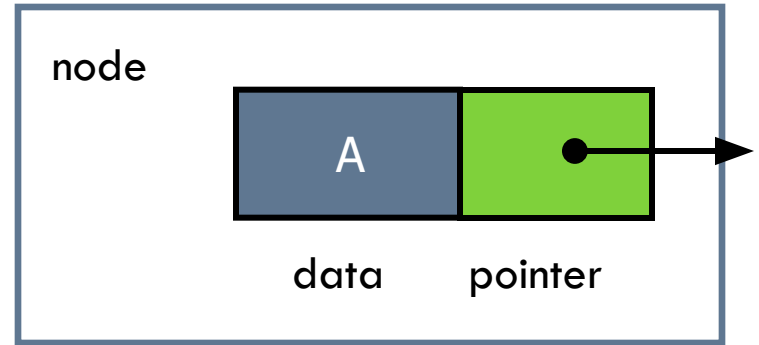
Elements in a linked list are **linked using pointers** as shown

- A list is a collection of items that has a particular order
 - It can have an **arbitrary** length
 - Objects / elements can be **inserted or removed** at **arbitrary locations** in the list
 - A list can be traversed in order one item at a time

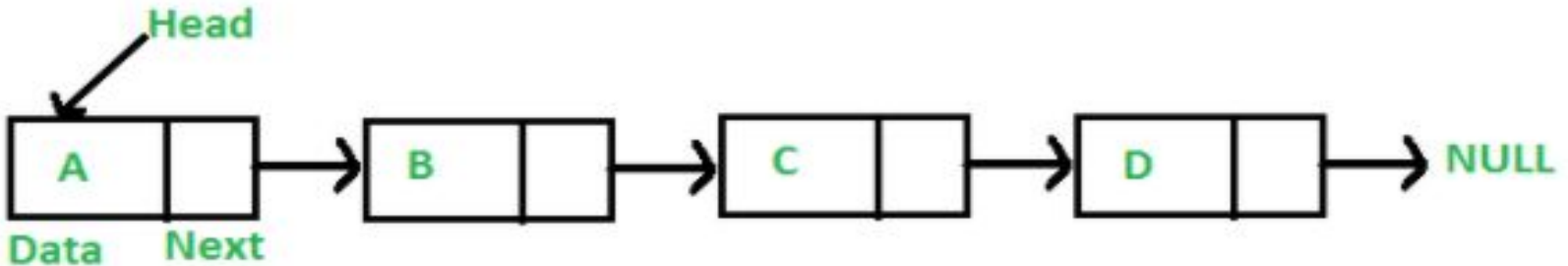


Linked List

- A **linked list** is a series of connected **nodes**
- Each node contains at least
 - A piece of data (any type)
 - Pointer to the next node in the list
- **Head:** pointer to the first node
- The last node points to NULL



Head



Linked List Terminologies

□ **Traversal of List**

- Means to visit every element or node in the list beginning from first to last.

□ **Predecessor and Successor**

- In the list of elements, for any location n , $(n-1)$ is predecessor and $(n+1)$ is successor.
- In other words, for any location n in the list, the **left** element is **predecessor** , **right** element is **successor**.
- Also, the first element does not have predecessor and the last element does not have successor.

Arrays Vs Linked Lists

- Arrays are lists that have a **fixed** size in memory. **int stud [5]**
- The programmer must keep **track** of the **length** of array
- No matter **how many elements** of the array are used in a program, the array has **same amount of allocated space**.
- Array elements are stored in **successive memory** locations. Also, **order** of elements stored in array is **same logically** and **physically**.
- 1001 1003 1005 1007 1009

Array Vs **Linked Lists**

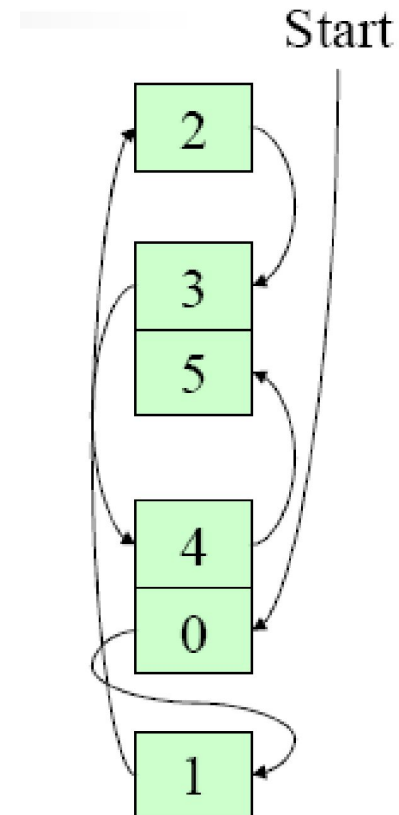
- A linked list takes up only **as much space in memory as** is needed for the **length** of the **list**. Stud 10
- List **expands** or **contracts** as you add or delete elements.
- In linked list the elements are **not stored in successive** memory location
- Elements can be **added** to (or **deleted** from) **either end**, or added to (or deleted from)the **middle** of the list.

Array Vs **Linked Lists**

- Linked lists are **more complex** to code and **manage** than arrays, but they have some distinct **advantages**.
 - **Dynamic**: a linked list can easily **grow** and **shrink** in size.
 - We don't need to know how many nodes will be in the list. They are created in memory as needed.
 - In contrast, the size of a C++ array is fixed at compilation time.
 - **Easy and fast insertions and deletions**
 - To **insert** or **delete** an element in an **array**, we need to **copy** to **temporary variables** to make **room for new elements** or **close the gap caused by deleted elements**.
 - With a linked list, **no need to move other nodes**. Only need to **reset** some **pointers**.

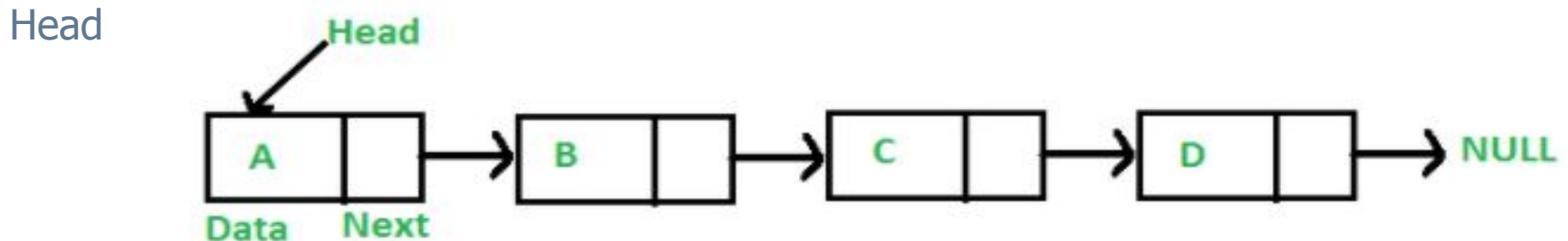
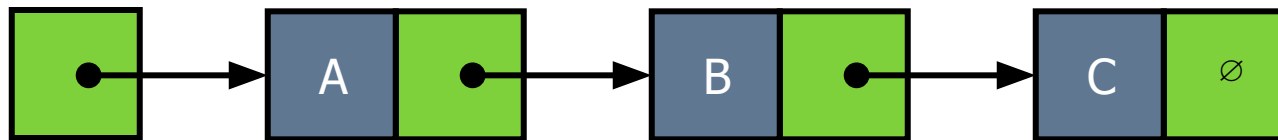
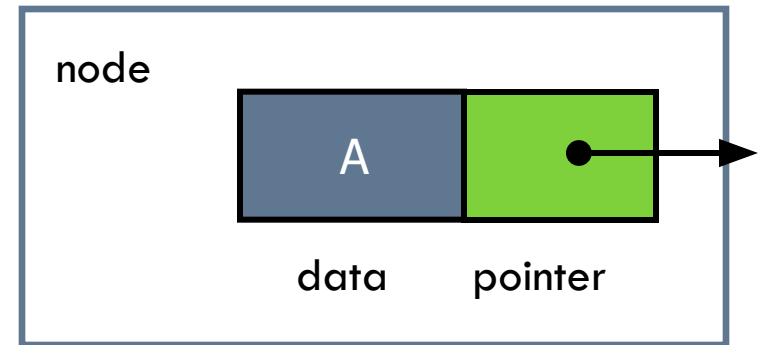
An Array

A Linked List



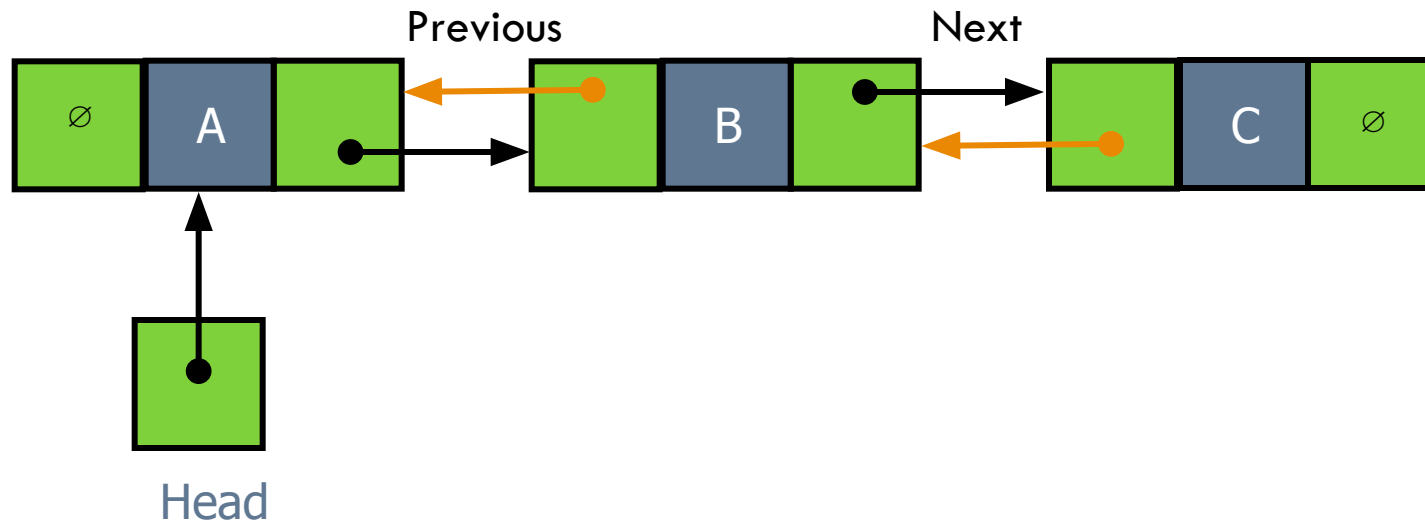
Types of Linked List - *Singly linked lists*

- A **linked list** is a series of connected **nodes**
- Each node contains at least
 - A piece of data (any type)
 - Pointer to the next node in the list
- **Head:** pointer to the first node
- The last node points to NULL



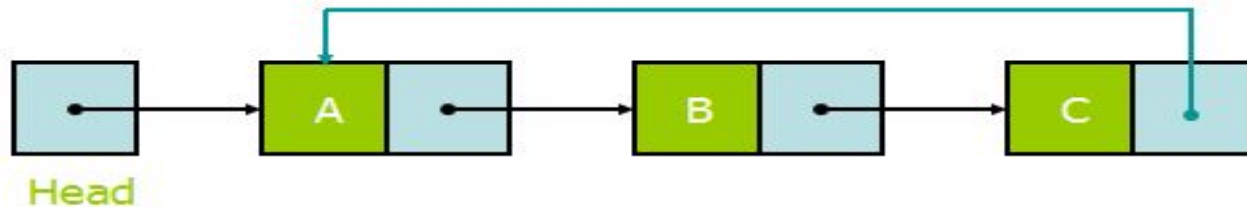
Types - Doubly linked lists

- Each node points to not only successor but Predecessor
- There are two NULL: at the first and last nodes in the list
- Advantage: given a node, it is easy to visit its predecessor.
- Convenient to traverse lists **backwards**

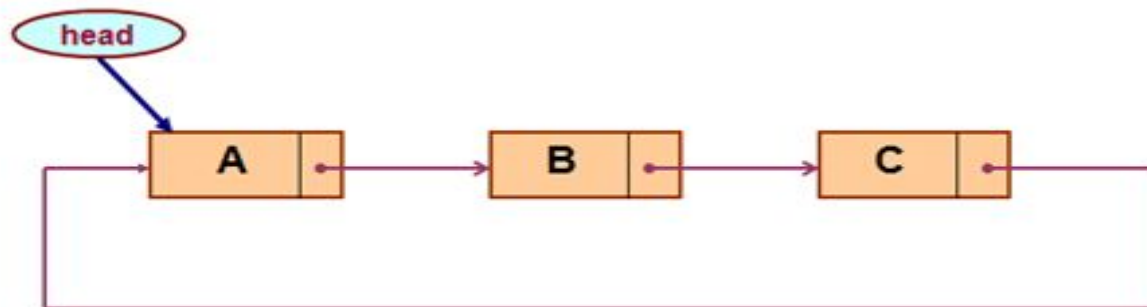


Types - Circular linked lists

- The last node points to the first node of the list



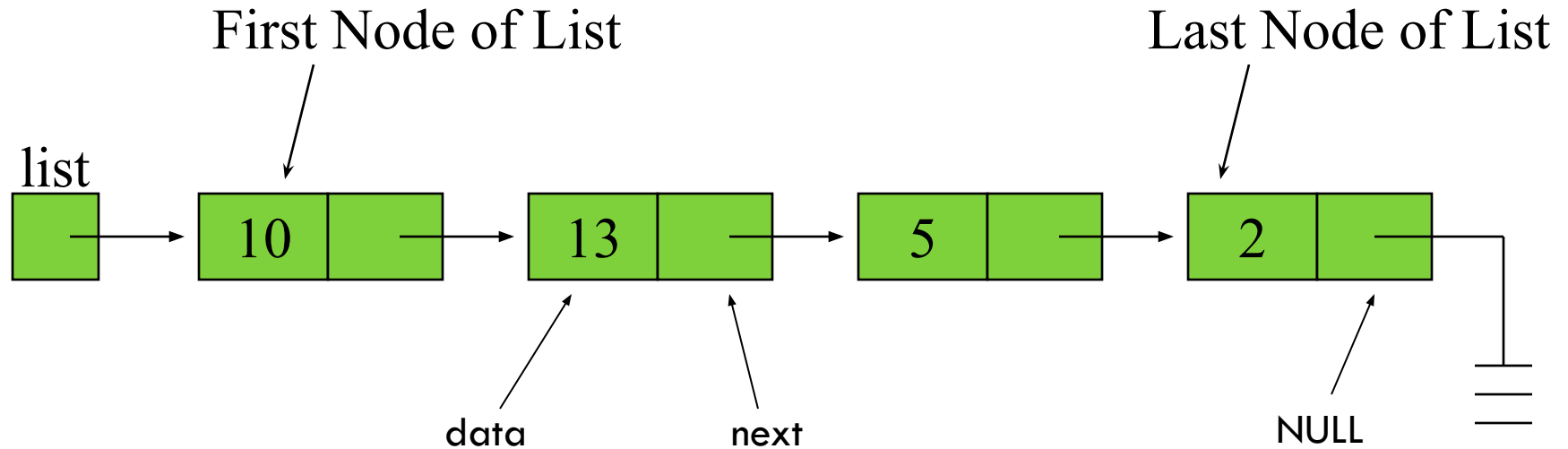
- How do we know when we have finished traversing the list?
- Ans - check if the pointer of the current node is equal to the head.
- The pointer from the last element in the list points back to first element.



Basic **Operations** of Linked List

- Operations of Linked List
 - IsEmpty: **determine** whether or not the list is empty
 - InsertNode: **insert** a new node at a particular position
 - FindNode: **find** a node with a given value
 - DeleteNode: **delete** a node with a given value
 - DisplayList: **print** all the nodes in the list

An integer linked list



Creating a List node

```
struct Node{  
    int data;        // data in node  
    Node *next;      // Pointer to next node  
};
```

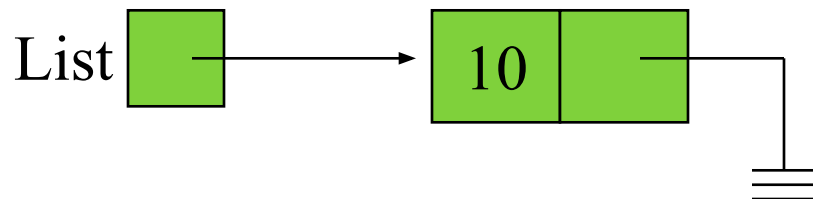
```
Node *list;
```

```
list= NULL
```

```
list= new Node;
```

```
list -> data = 10;           Node.data =10
```

```
list -> next = NULL;        Node.next ---- not possible
```



The NULL pointer

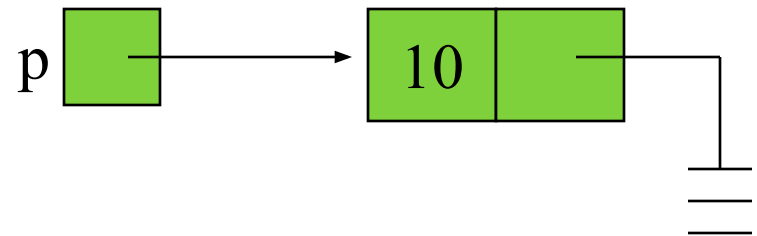
- ❑ NULL is a special pointer value that does not reference any memory cell.
- ❑ If a pointer is not currently in use, it should be set to NULL so that one can determine that it is **not pointing to a valid address**:

```
int *p;  
p = NULL;
```

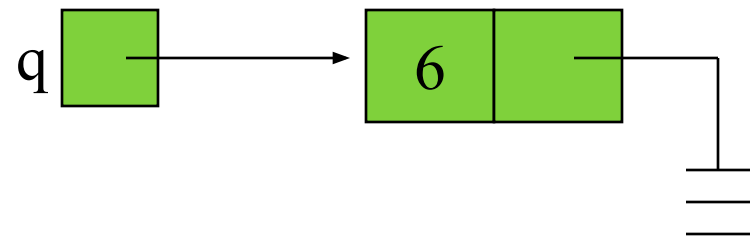
Adding a node to a list

Node *p, *q;

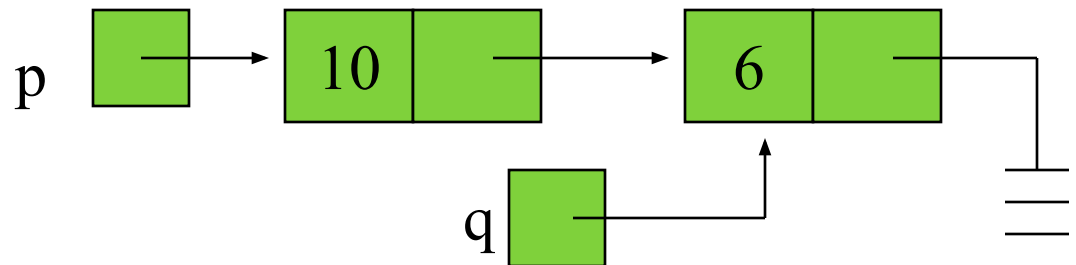
```
p = new Node;  
p -> data = 10;  
p -> next = NULL;
```



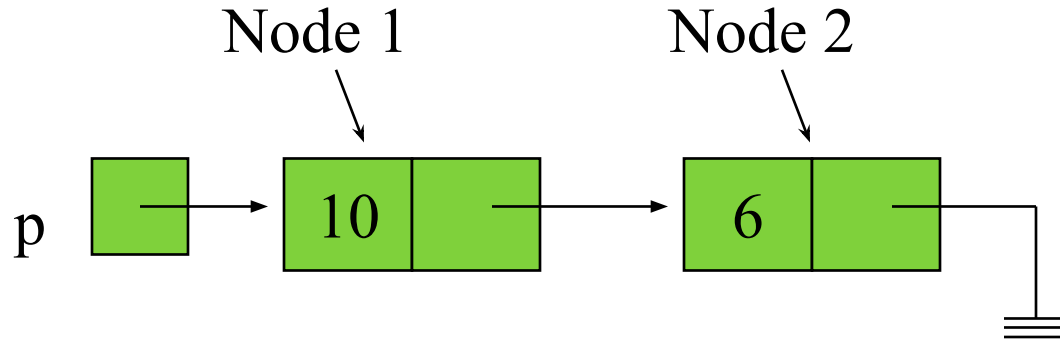
```
q = new Node;  
q -> data = 6;  
q -> next = NULL;
```



```
p -> next = q;
```



Accessing List Data



Expression

Value

p

Pointer to first node (head)

p -> data

10

p -> next

Pointer to next node

p -> next -> data

6

p -> next -> next

NULL pointer

Using typedef with pointers

```
struct Node {  
    int data;        // data in node  
  
    Node *next;      // Pointer to next node  
};
```

```
typedef Node *NodePtr; // NodePtr type is a pointer  
                      // to a Node  
  
Node *p;              // p is a pointer to a Node  
  
NodePtr q;             // q is a pointer to a Node
```

Building a list from 1 to n

```
struct Node {
```

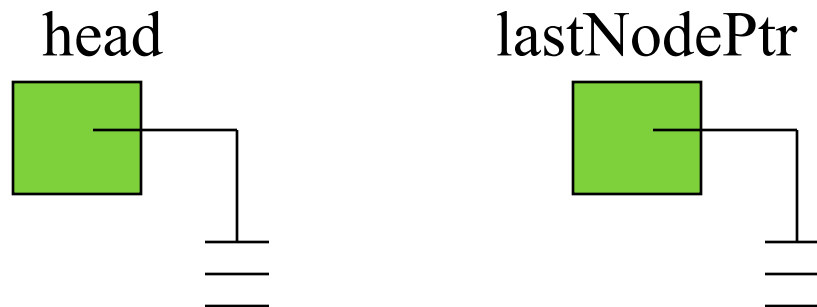
```
    int data;
```

```
    Node *next;
```

```
};
```

```
Node *head = NULL;    // pointer to the list head
```

```
Node *lastNodePtr = NULL; // pointer to last node in list
```



Creating the first node

```
Node *ptr;           // declare a pointer to Node
```

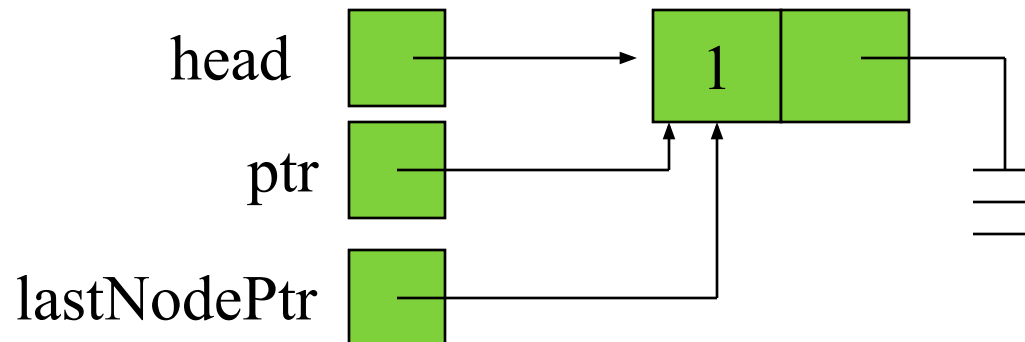
```
ptr = new Node;      // create a new Node
```

```
ptr -> data = 1;
```

```
ptr -> next = NULL;
```

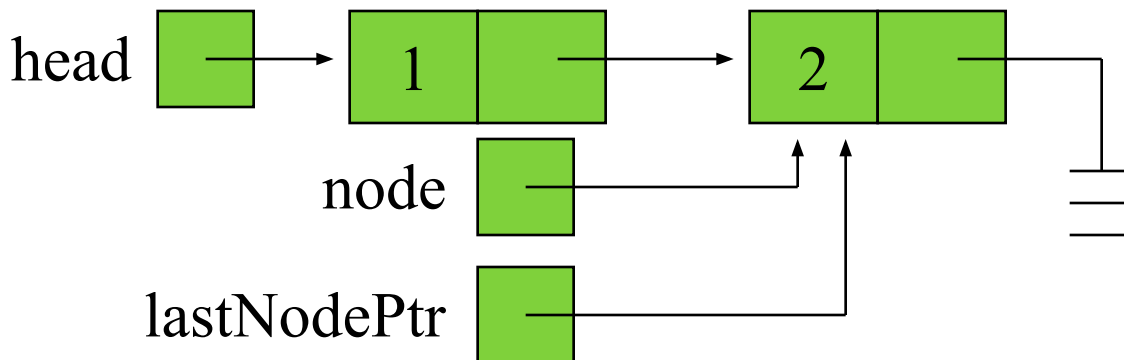
```
head = ptr;          // new node is first
```

```
lastNodePtr = ptr;  // and last node in list
```

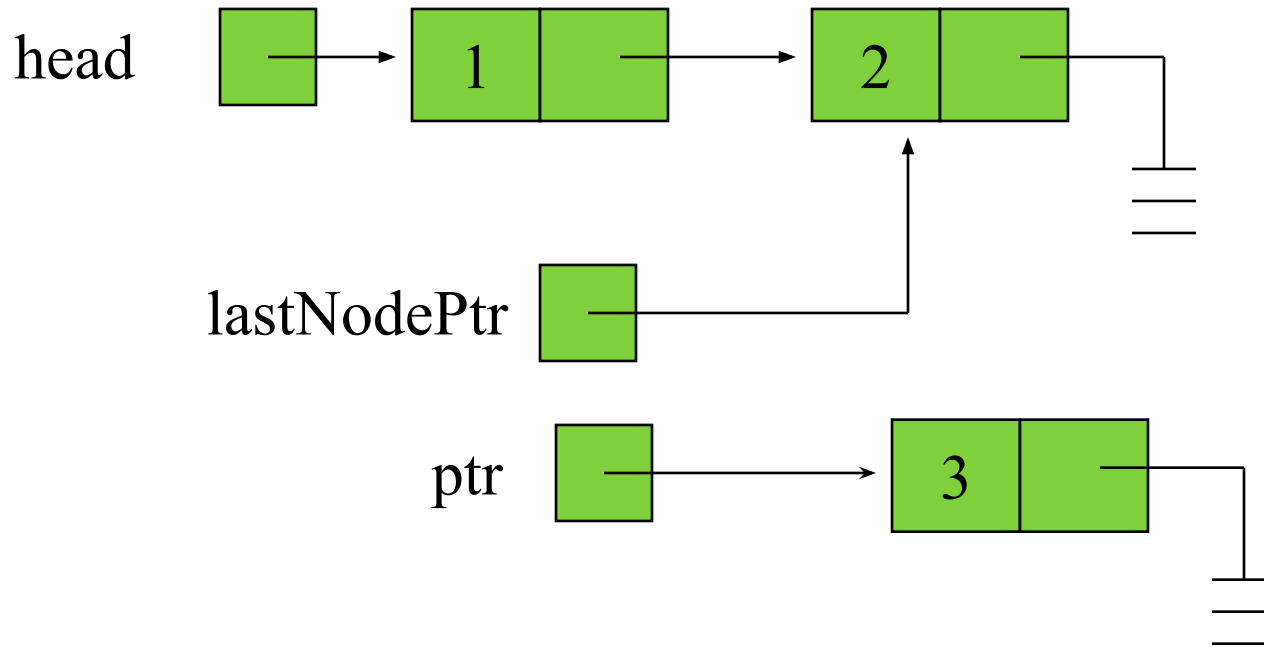
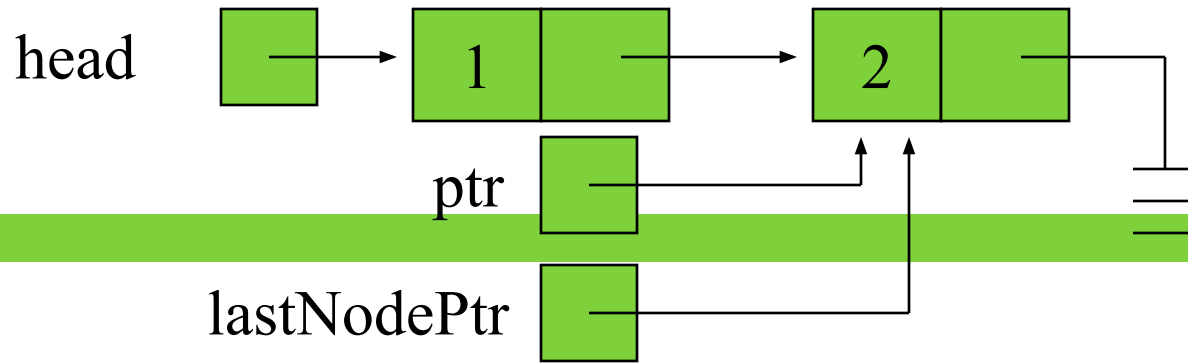


Adding more nodes

```
for (int i = 2; i <= n; i ++ ) {  
    ptr = new Node;    //create new node  
  
    ptr -> data = i;  
  
    ptr -> next = NULL;  
  
    lastNodePtr -> next = ptr;    // order is  
  
    lastNodePtr = ptr;    // important  
}
```

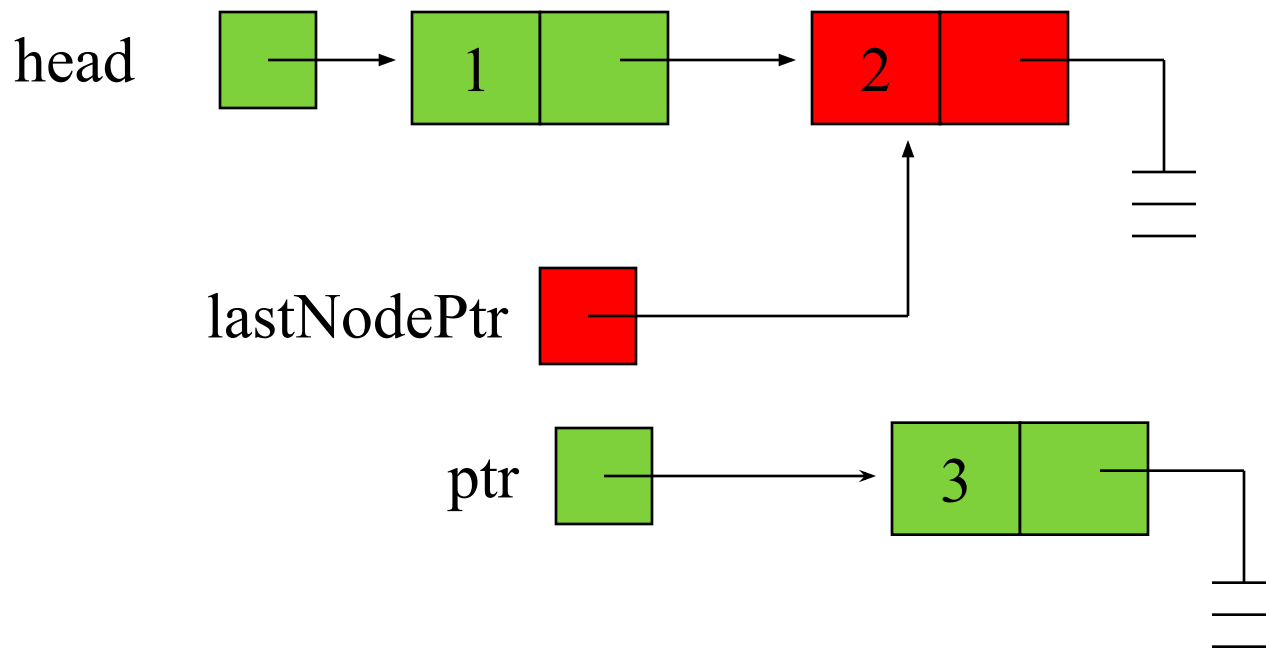
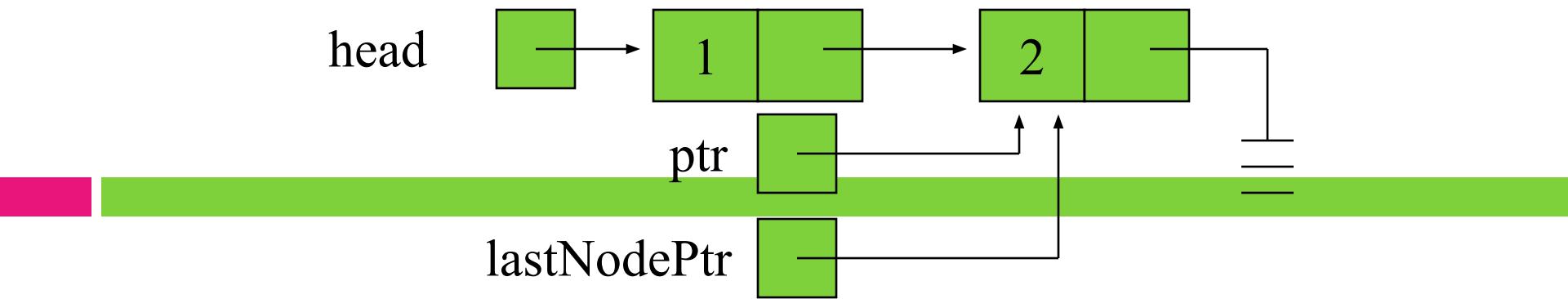


Initially

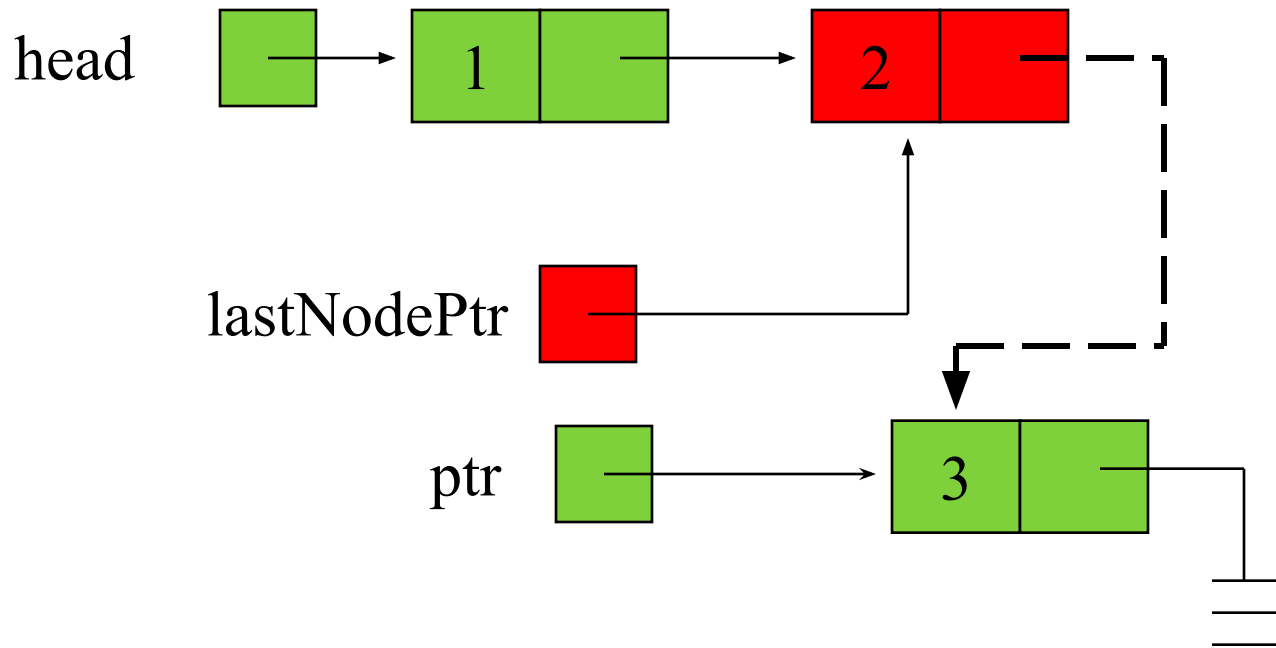
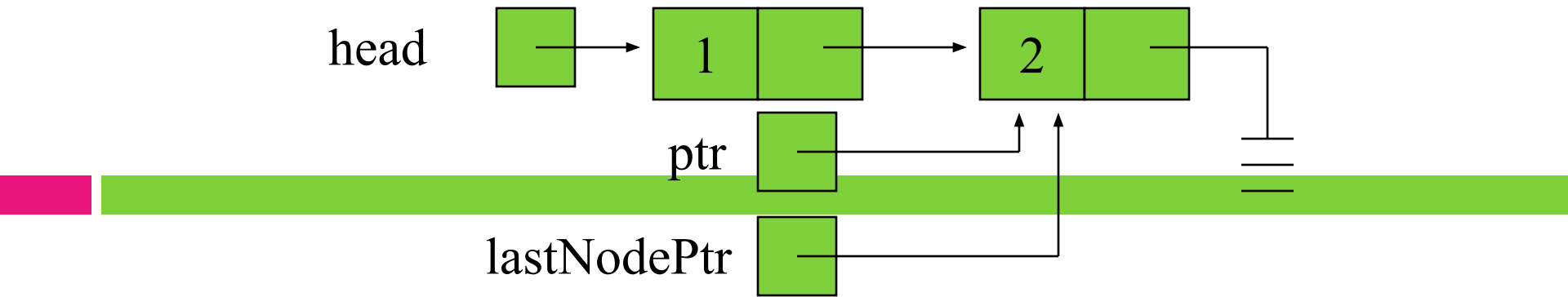


Create a new node with data field set to 3

Its next pointer should point to NULL

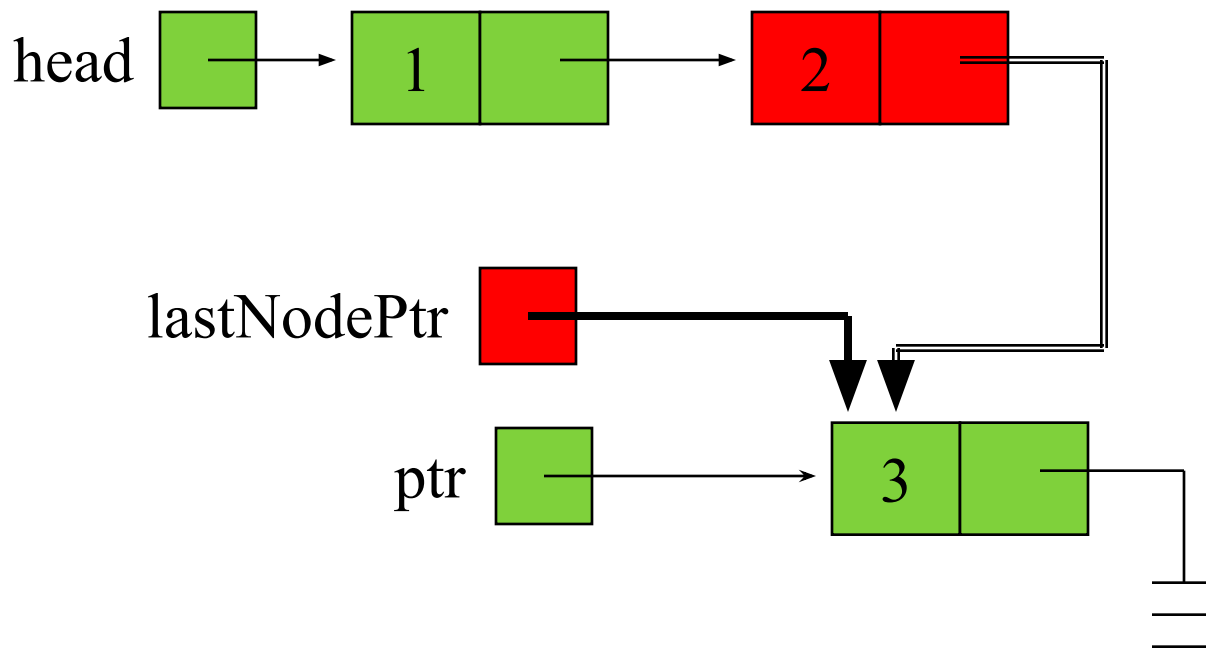
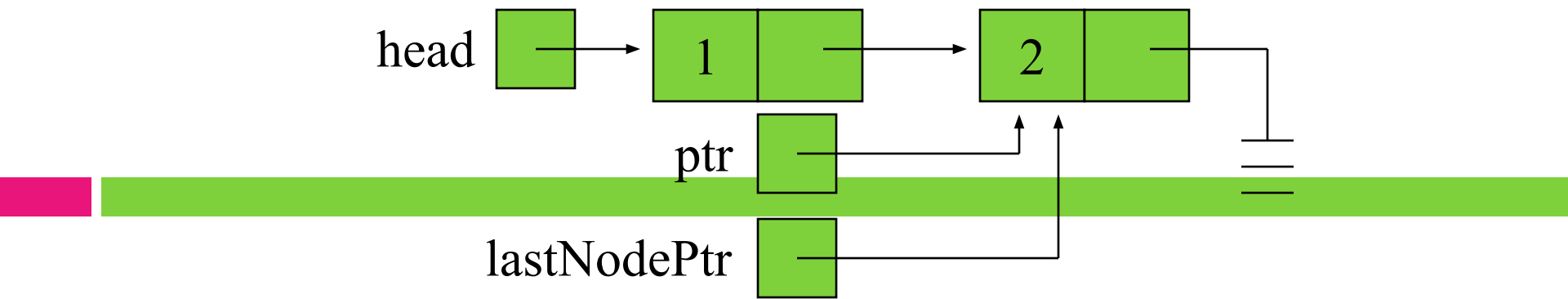


The next pointer of the node which was previously last should now point to newly created node “`lastNodePtr->next=ptr`”



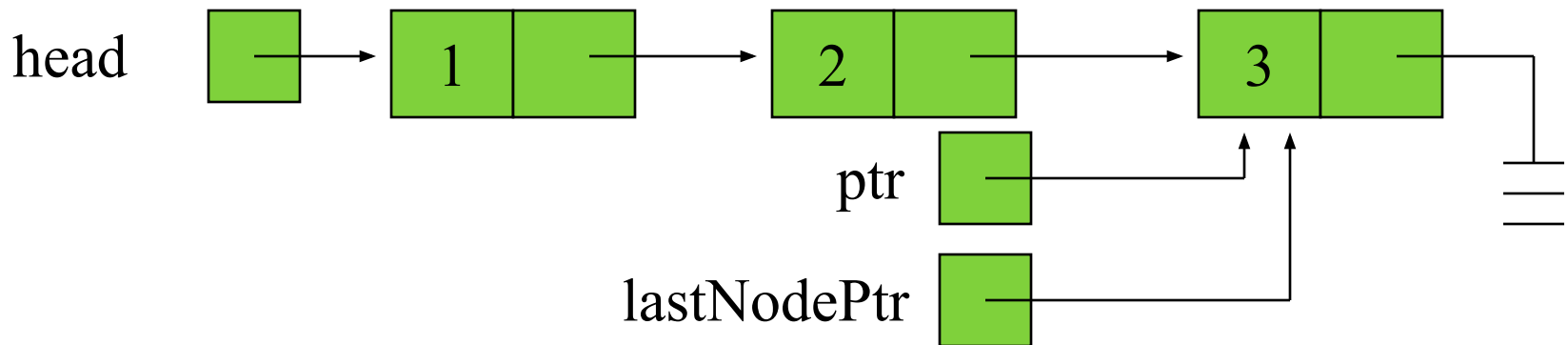
The next pointer of the node which was previously last should now point to newly created node “**lastNodePtr->next=ptr**”

LastNodePtr should now point to the newly created Node “lastNodePtr = ptr;”

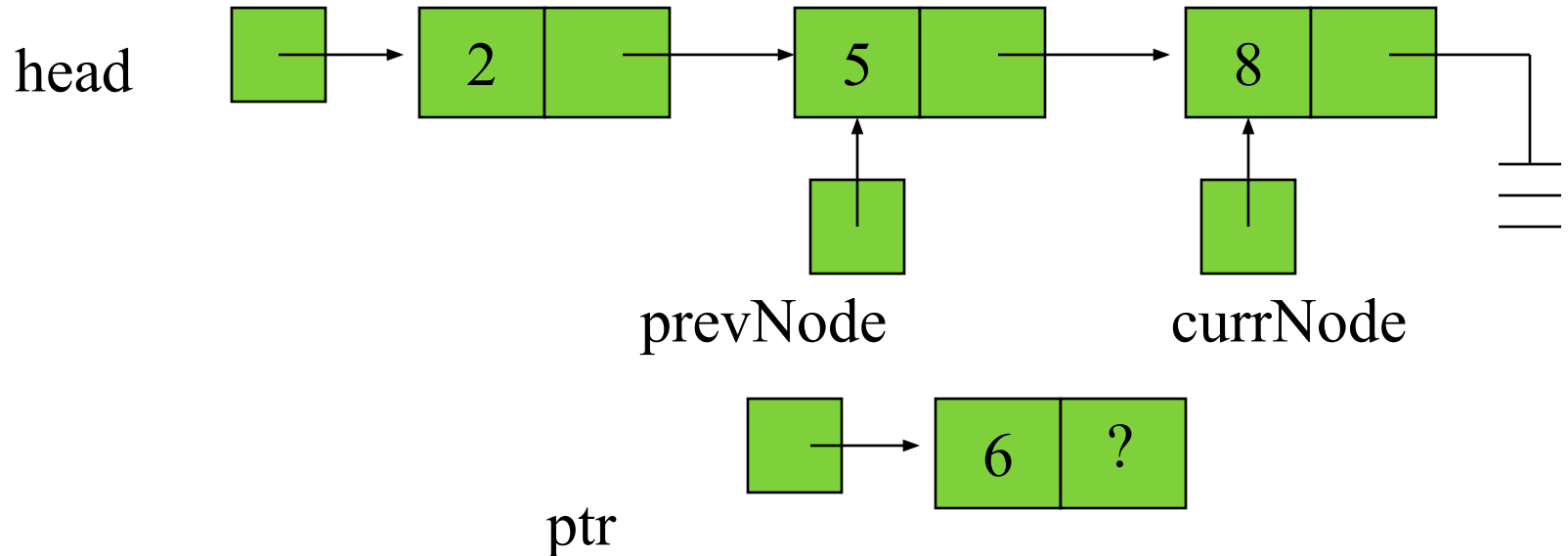


LastNodePtr should now point to the newly created Node “**`lastNodePtr = ptr;`**”

Re-arranging the view



Inserting a node in a list



Step 1: Determine where you want to insert a node.

Step 2: Create a new node:

```
Node *ptr;
```

```
ptr = new Node;
```

```
ptr -> data = 6;
```

```
Node *ptr, *currNode, *prevNode ;
prevNode = head;
ptr = new Node;
ptr->data = 6;
ptr->next = NULL;
currNode = head->next;
while (currNode->data < ptr->data)
{
    prevNode = currNode;
    currNode = currNode->next;
}
```

Note:

when this loop terminates **prevNode** and **currNode** are at a place where insertion will take place. Only the “LINKS” or pointers of the list remain to be adjusted

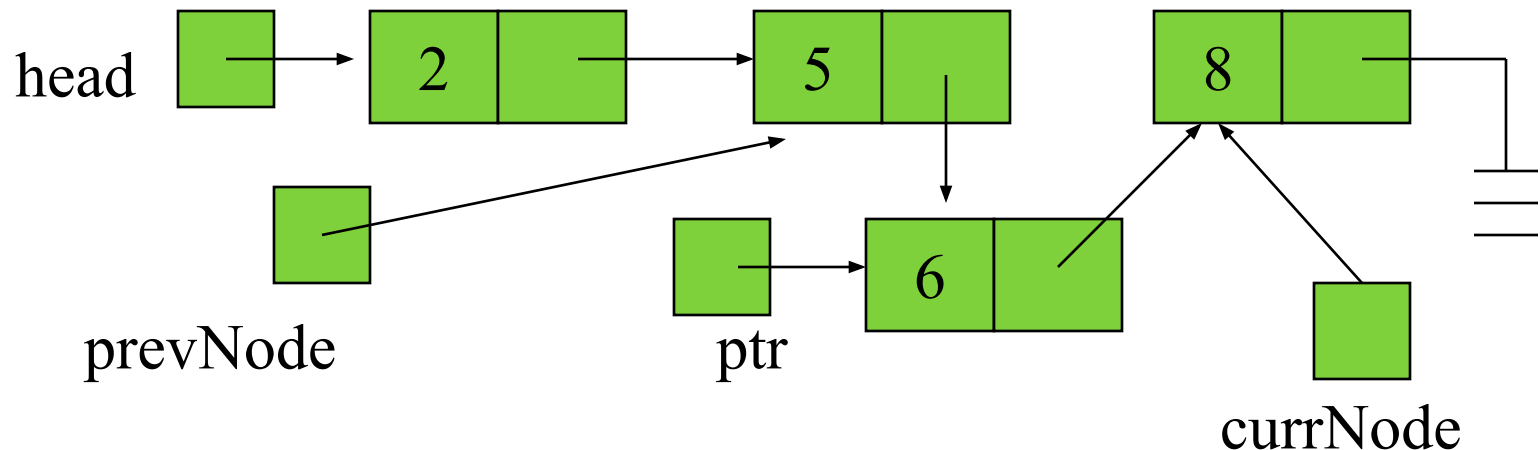
Continuing the insert

Step 3: Make the new node point to the current Node pointer.

`ptr -> next = currNode;`

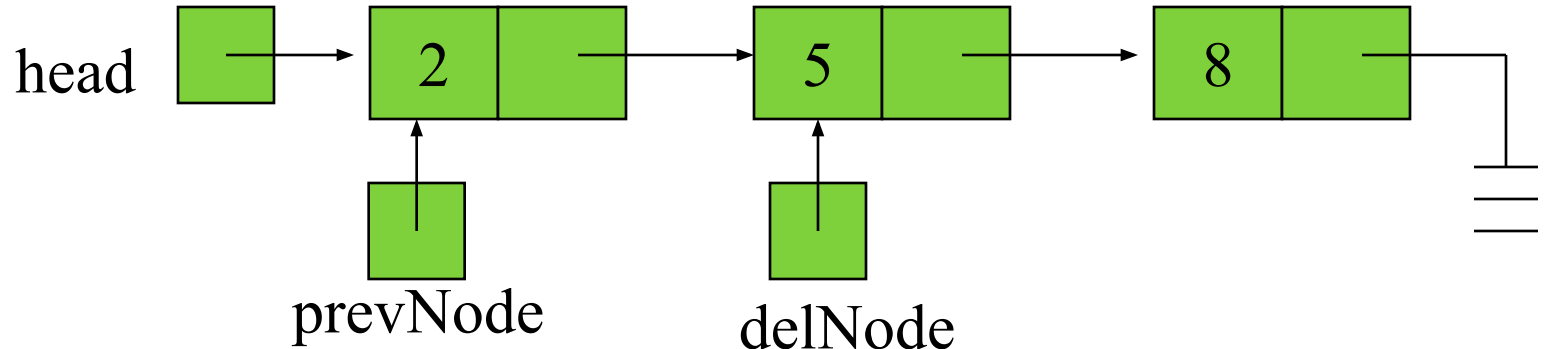
Step 4: Make previous node point to the new node:

`prevNode -> next = ptr;`



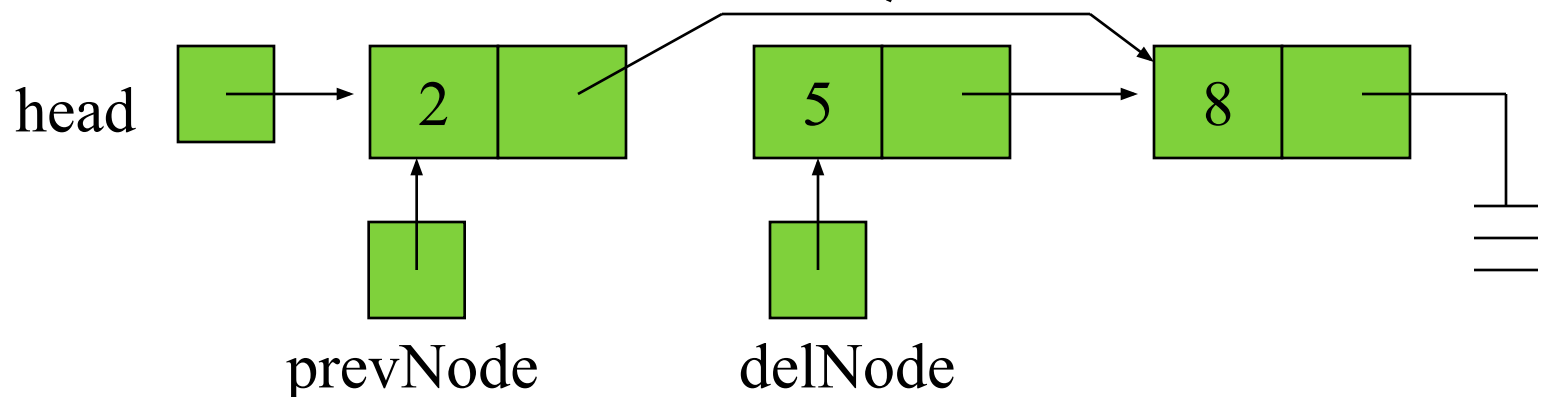
Now The new link has been added in the linked list

Deleting a node from a list



Step 1: Redirect pointer from the Node before the one to be deleted to point to the Node after the one to be deleted.

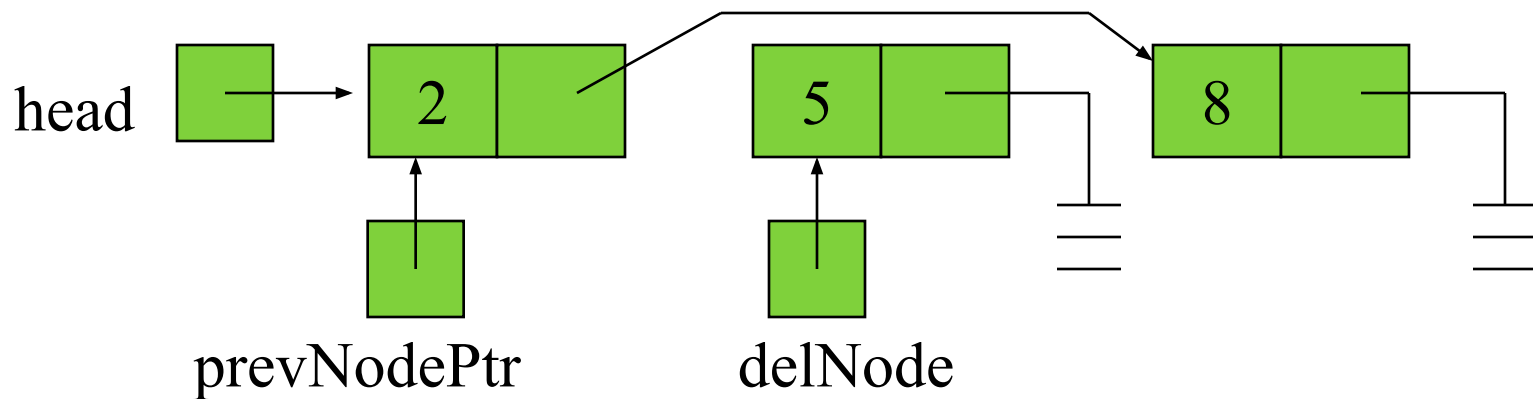
`prevNode -> next = delNode -> next;`



Finishing the deletion

Step 2: Remove the pointer from the deleted link.

```
delNode -> next = NULL;
```



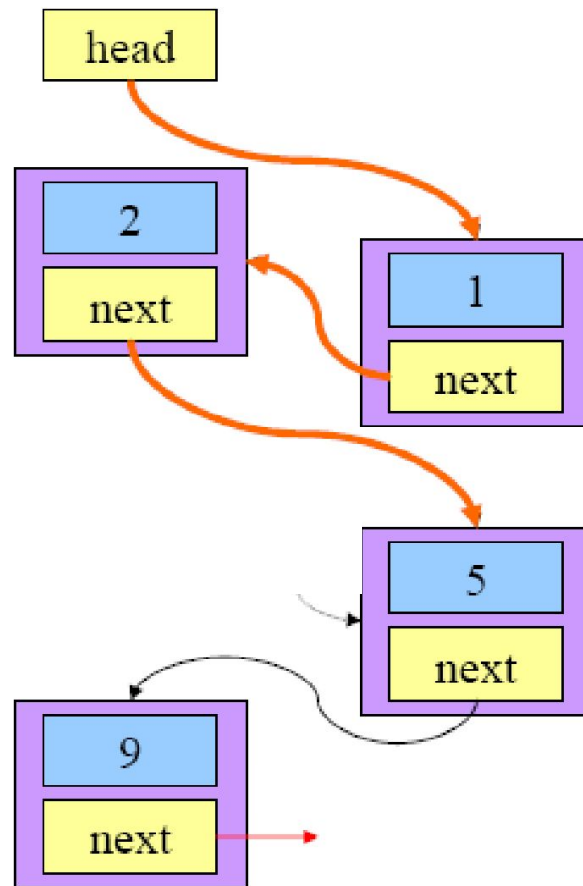
Step 3: Free up the memory used for the deleted node:

```
delete delNode;
```

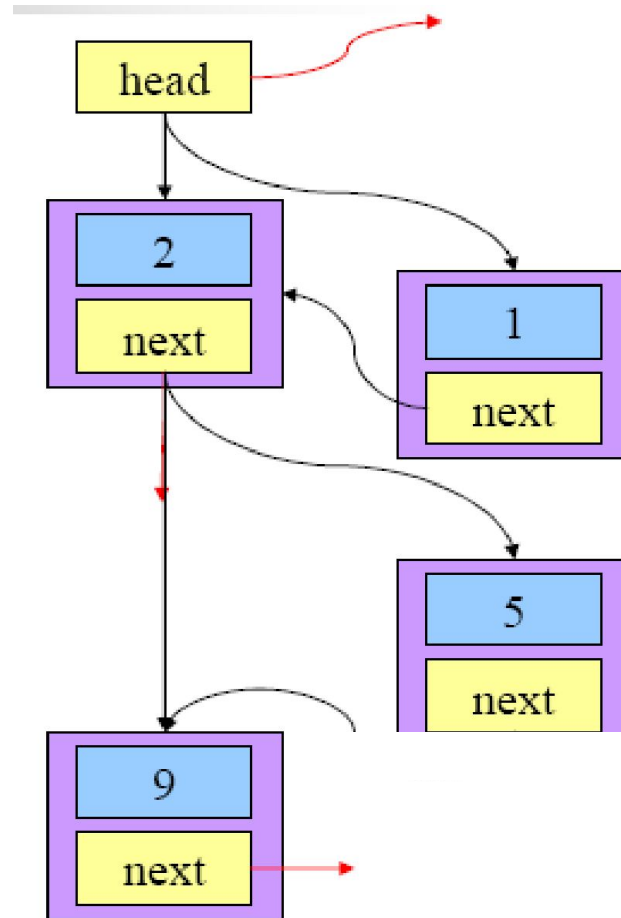


List Operations - Summarized

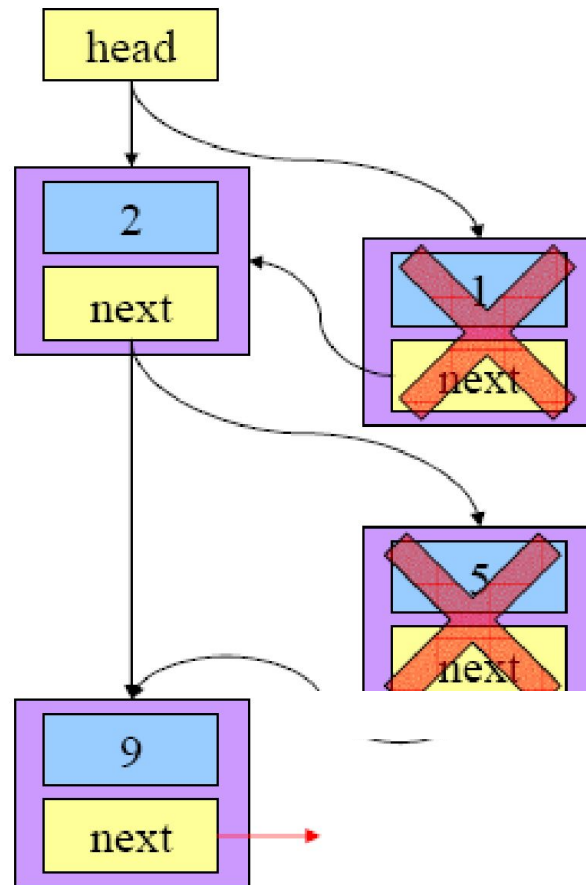
Traversing a Linked List



Insertion in a Linked List

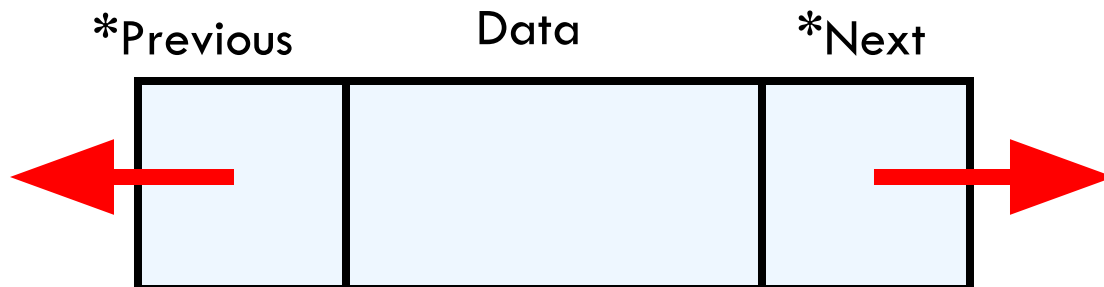


Deletion from a Linked List



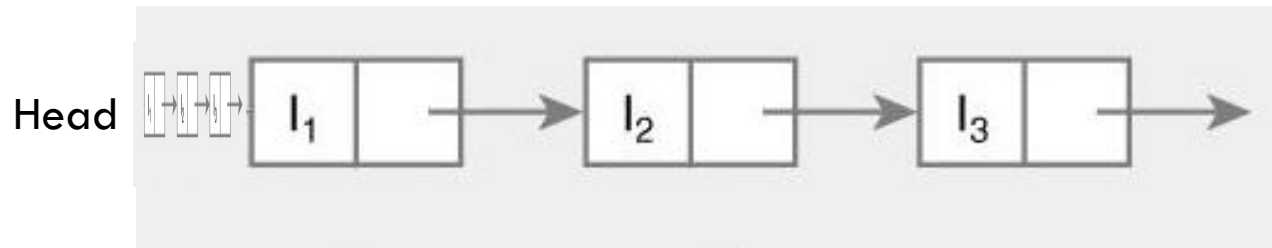
Doubly Linked List

- The singly linked list contains only one pointer field i.e. every node holds an address of next node.
- The singly linked list is uni-directional i.e. we can only move from one node to its successor.
- This limitation can be overcome by **Doubly linked list**.
- In Doubly linked list, each node has two pointers.
- One pointer to its successor (NULL if there is none) and one pointer to its predecessor (NULL if there is none).
- These pointers enable **bi-directional traversing**

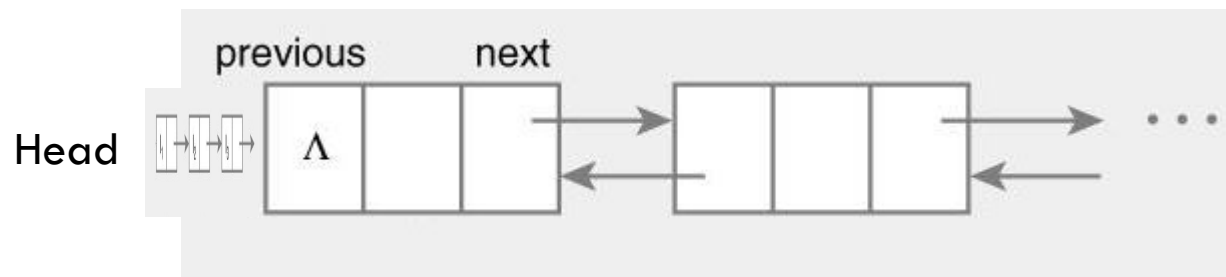


Comparison of Linked Lists

A Singly Linked List



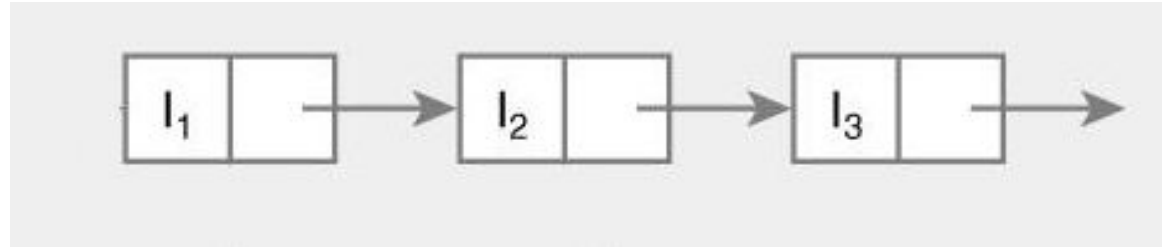
A Doubly Linked List



Comparison of Linked Lists

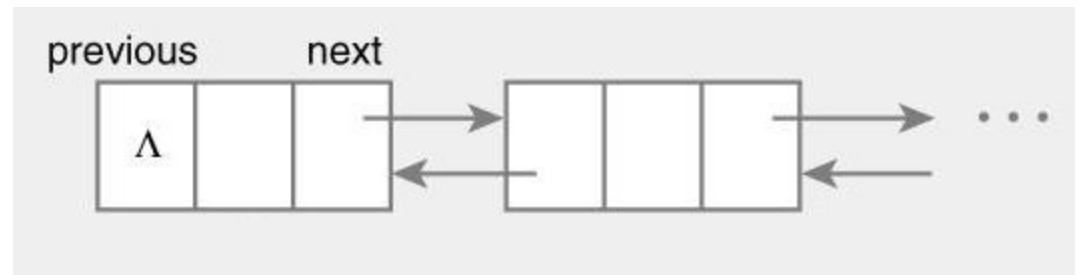
□ Linked list

```
struct Node {  
    int data;  
    Node* next;  
};
```



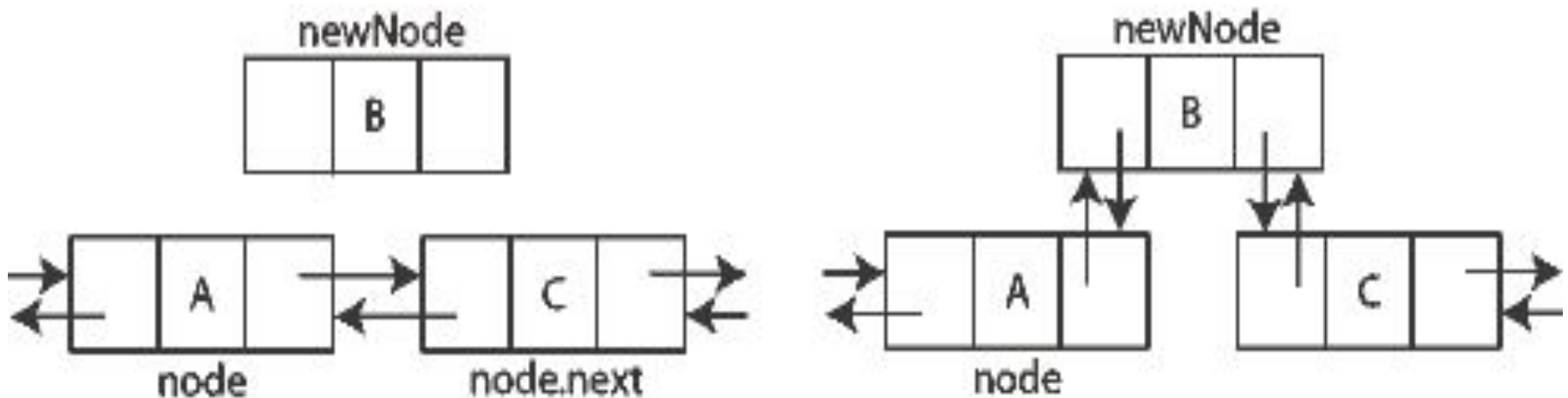
□ Doubly linked list

```
struct Node {  
    Node *previous;  
    int data;  
    Node *next;  
};
```



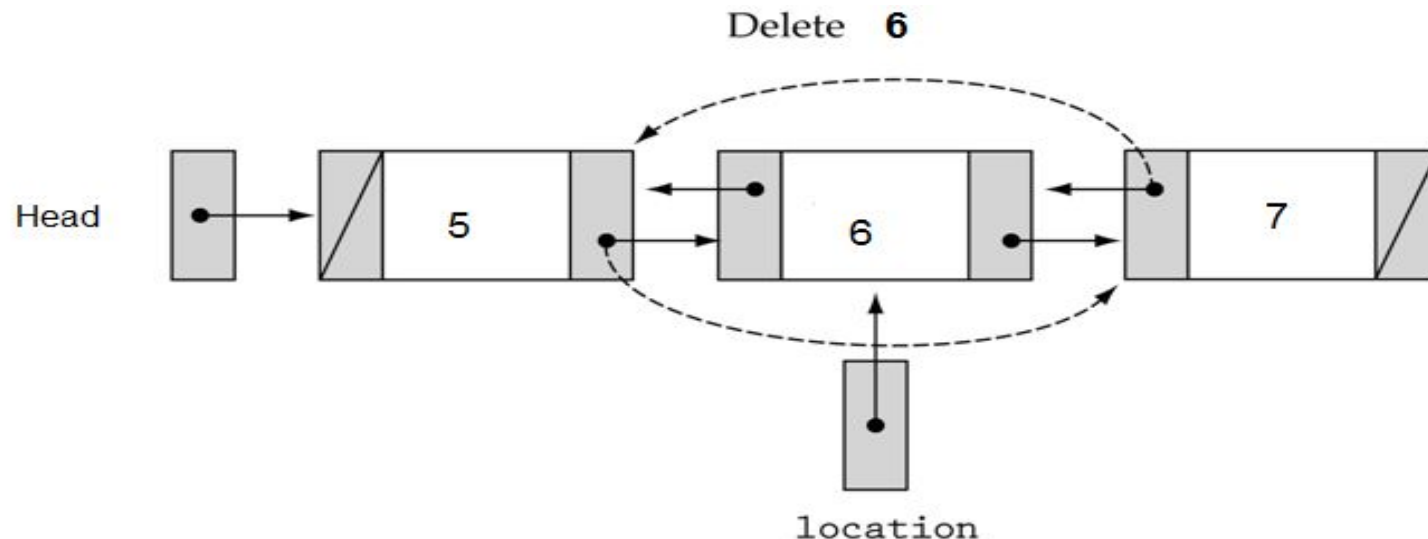
Insertion

- In insertion process, element can be inserted in three different places
 - At the beginning of the list
 - At the end of the list
 - At the specified position.
- To insert a node in doubly linked list, you must **update pointers** in both **predecessor** and **successor** nodes.



Deletion

- In deletion process, element can be deleted from three different places
 - From the beginning of the list
 - From the end of the list
 - From the specified position in the list.
- When the node is deleted, the memory allocated to that node is released and the previous and next nodes of that node are linked



Doubly Linked List-

Advantages & Disadvantages

Advantages:

- ❑ The doubly linked list is **bi-directional**, i.e. it can be traversed in both **backward** and **forward** direction.
- ❑ The **operations** such as insertion, deletion and searching can be done from **both** ends.
- ❑ **Predecessor** and **successor** of any element can be searched quickly

Disadvantages:

- ❑ It consume more memory space.
- ❑ There is a **large pointer adjustment** during insertion and deletion of element.
- ❑ It consumes more time for few basic list operations

Singly Linked List **Application-** **Polynomial Representation and Addition**

- A **Polynomial** is an expression that contains more than two terms. A term is made up of coefficient and exponent. An example of polynomial is

$$P(x) = 4x^3 + 6x^2 + 7x + 9$$

- Polynomial thus may be represented using arrays or linked lists.
- Array representation assumes that the exponents of the given expression are arranged from 0 to the highest value (degree), which is represented by the subscript of the array beginning with 0. The coefficients of the respective exponent are placed at an appropriate index in the array. The array representation for the above polynomial expression is given below:

arr	9	7	6	4	(coefficients)
	0	1	2	3	(exponents)

$$P(x) = 4x^3 + 6x^2 + 7x + 9$$

$$Q(x) = 8x^3 + 7x^2 + 3x + 2$$

Singly Linked List **Application-** **Polynomial Representation and Addition**

Polynomial may also be represented using a linked list. Structure may be defined such that it contains two parts- one is the coefficient and second is the corresponding exponent. The structure definition may be given as shown below:

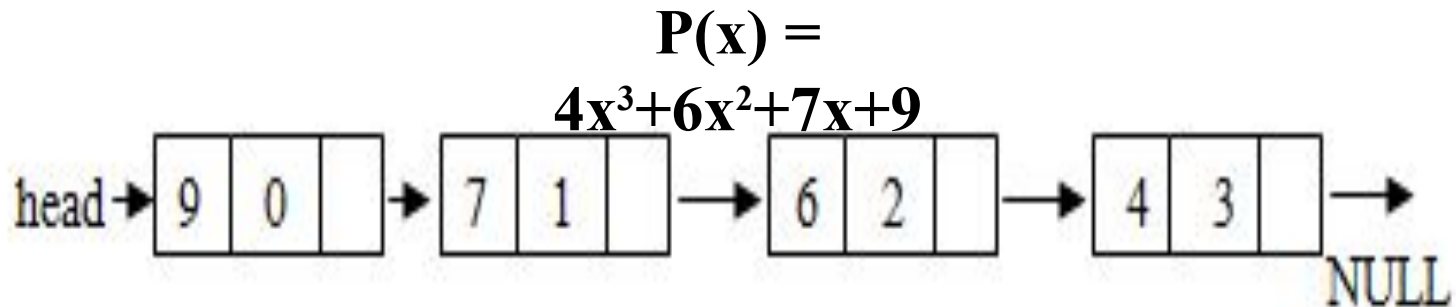
coef	expon	link
------	-------	------

```
struct polynomial
{
    int coefficient;
    int exponent;
    struct polynomial *next;
};
```

Singly Linked List **Application-** **Polynomial Representation and Addition**

47

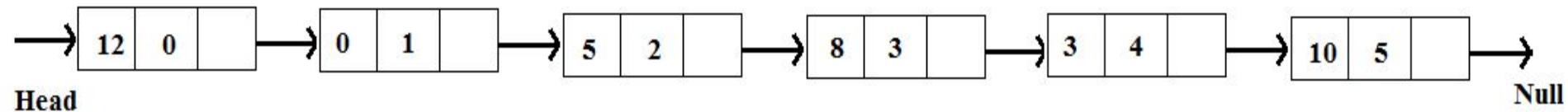
- Thus the above polynomial may be represented using linked list as shown below:



$$\begin{array}{ccc} Q(x) = & & \\ 8x^3 + 7x^2 + 3x + 2 & \xrightarrow{\quad} & R(x) = \\ R(x) = & & R(x) = \\ x^3 + 3x + 2 & & x^3 + 0x^2 + 3x + 2 \end{array}$$

Exercise

- Identify the polynomial



- $10x^5 + 3x^4 + 8x^3 + 5x^2 + x + 12$
- $5x^{10} + 3x^4 + 8x^3 + 5x^2 + x + 12$
- $10x^5 + 3x^4 + 8x^3 + 5x^2 + 12$
- $5x^{10} + 4x^3 + 8x^3 + 5x^2 + 12$

$$P(x) = 5x^2 + 6x + 12$$

$$Q(x) = 2x^2 + 9x + 5$$

$$R(x) = 7x^2 + 15x + 17$$

Polynomials [cont...]

Addition of two Polynomials:

- For adding two polynomials using arrays is straightforward method, since both the arrays may be added up element wise beginning from 0 to $n-1$, resulting in addition of two polynomials.
- Addition of two polynomials using linked list requires comparing the exponents, and wherever the exponents are found to be same, the coefficients are added up. For terms with different exponents, the complete term is simply added to the result thereby making it a part of addition result.

Multiplication of two Polynomials:

- Multiplication of two polynomials however requires manipulation of each node such that the exponents are added up and the coefficients are multiplied.
- After each term of first polynomial is operated upon with each term of the second polynomial, then the result has to be added up by comparing the exponents and adding the coefficients for similar exponents and including terms as such with dissimilar exponents in the result.

Polynomials [cont...]

Attach a node to the end of a list

```
void attach(float coefficient, int exponent, poly_pointer *ptr){
/* create a new node with coef = coefficient and expon = exponent, attach
it to the node pointed to by ptr.  Ptr is updated to point to this new
node
*/
poly_pointer temp;
temp = (poly_pointer) malloc(sizeof(poly_node));
/* create new node */
if (IS_FULL(temp)) {
    fprintf(stderr, "The memory is full\n");
    exit(1);
}
temp->coef = coefficient;    /* copy item to the new node */
temp->expon = exponent;
(*ptr)->link = temp; /* attach */
*ptr = temp;    /* move ptr to the end of the list */
```

Polynomials [cont...]

Analysis of PADD (Polynomial addition)

$$A(x) = \sum_{i=0}^{m-1} a_i x^i + \dots + a_{m-1} x^{m-1} \quad B(x) = \sum_{j=0}^{n-1} b_j x^j + \dots + b_{n-1} x^{n-1}$$

1. coefficient additions

$$0 \leq \text{additions} \leq \min(m, n)$$

where m (n) denotes the number of terms in A (B).

2. exponent comparisons extreme case:

$$e_{m-1} > f_{m-1} > e_{m-2} > f_{m-2} > \dots > e_1 > f_1 > e_0$$

$> f_0$ $m+n-1$ comparisons

3. creation of new nodes extreme case: maximum number of terms in d is $m+n$ $m+n$ new nodes

summary: $O(m+n)$

Polynomials [cont...]

- A Suite for Polynomials

$$e(x) = a(x) * b(x) + d(x)$$

poly_pointer a, b, d, e;

...

a = read_poly();

b = read_poly();

temp = pmult(a, b);

e = padd(temp, d);

print_poly(e);

read_poly()

print_poly()

padd()

psub()

pmult()

temp is used to hold a partial result.
By returning the nodes of temp, we
may use it to hold other polynomials

Polynomials [cont...]

- Erase Polynomials: erase frees the nodes in *temp*

```
void erase (poly_pointer *ptr){  
    /* erase the polynomial pointed to by ptr */  
    poly_pointer temp;  
    while ( *ptr){ temp = *ptr;  
        *ptr = (*ptr) -> link; free(temp);  
    }  
}
```

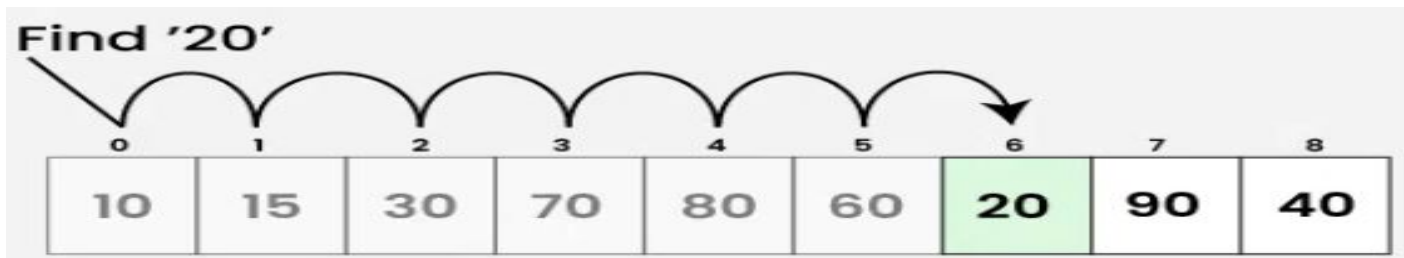
Topics

54

- Sorting Techniques - bubble sort, selection sort, insertion sort & their Analysis.
- Linked List:
 - Introduction, Representation of Linked List, Memory Allocation, Linked List v/s Array,
 - Types of Linked List - Singly Linked List, Circularly Linked Lists, Two-Way Lists (or Doubly Linked Lists),
 - Operations on Singly Linked List and Doubly Linked List: - Insertion, Deletion, Traversing.
 - Advantages, Disadvantages, Applications of Arrays and Linked List.
 - Singly Linked List Application-Polynomial Representation and Addition.
- **Searching Techniques:**
 - Linear Search
 - Binary Search,
 - Hashing, Hash Functions, properties of good hash function, types of Hash Functions, Collision resolution Techniques.

Linear Search (Sequential Search)

- Each element of the given list is visited one by one in a sequential fashion to find the desired element.



Algorithm:

- **Start:** Begin at the first element of the collection of elements.
- **Compare:** Compare the current element with the desired element.
- **Found:** If the current element is equal to the desired element, return true or index to the current element.
- **Move:** Otherwise, move to the next element in the collection.
- **Repeat:** Repeat steps 2-4 until we have reached the end of collection.
- **Not found:** If the end of the collection is reached without finding the desired element, return that the desired element is not in the array.

Linear Search (Sequential Search)

Consider the array `arr[] = {10, 50, 30, 70, 80, 20, 90, 40}`
and `key = 30`

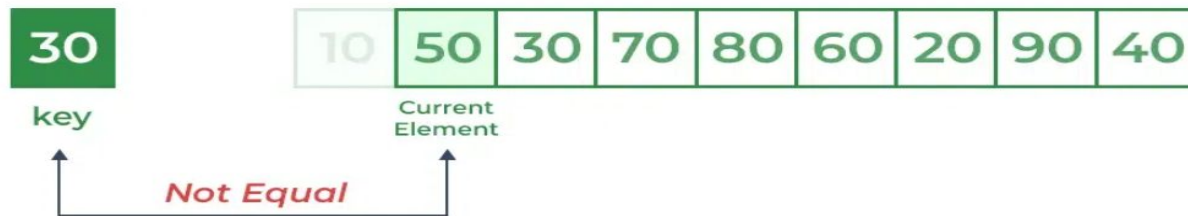
Step 1: Start from the first element (index 0) and compare `key` with each element (`arr[i]`).

Comparing `key` with first element `arr[0]`. Since not equal, the iterator moves to the next element as a potential match.



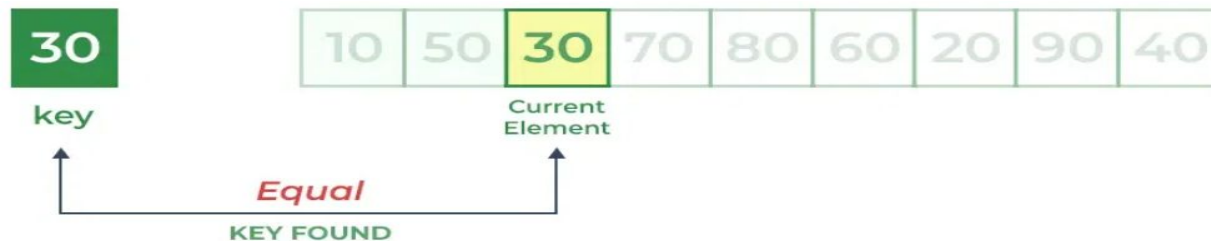
Linear Search (Sequential Search)

Comparing key with next element $\text{arr}[1]$. Since not equal, the iterator moves to the next element as a potential match.



Compare key with $\text{arr}[1]$

Step 2: Now when comparing $\text{arr}[2]$ with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).



Compare key with $\text{arr}[2]$

Linear Search-Time & Space Complexity

Time Complexity:

- **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is $O(1)$
- **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is $O(N)$ where N is the size of the list.
- **Average Case:** $O(N)$

Auxiliary Space: $O(1)$ as except for the variable to iterate through the list, no other variable is used.

Linear Search - Applications

- **Unsorted Lists:** When we have an unsorted array or list, linear search is most commonly used to find any element in the collection.
- **Small Data Sets:** Linear Search is preferred over binary search when we have small data sets with
- **Searching Linked Lists:** In linked list implementations, linear search is commonly used to find elements within the list. Each node is checked sequentially until the desired element is found.
- **Simple Implementation:** Linear Search is much easier to understand and implement as compared to Binary Search or Ternary Search.

When to use Linear Search Algorithm?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

Linear Search –

Advantages & Disadvantages

Advantages:

- Linear search can be used irrespective of whether the array is sorted or not. It can be used on arrays of any data type.
- Does not require any additional memory.
- It is a well-suited algorithm for small datasets.

Disadvantages:

- Linear search has a time complexity of $O(N)$, which in turn makes it slow for large datasets.
- Not suitable for large arrays.

Binary Search

- **Binary search** is a search algorithm used to find the position of a target value within a **sorted** array.
- It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty.
- The search interval is halved by comparing the target element with the middle value of the search space.
- Conditions while applying Binary Search algorithm:
 - The data structure must be sorted.
 - Access to any element of the data structure should take constant time.

Binary Search Algorithm

- Divide the search space into two halves by finding the middle index “mid”.
- Compare the middle element of the search space with the **key**.
- If the **key** is found at middle element, the process is terminated.
- If the **key** is not found at middle element, choose which half will be used as the next search space.
 - If the **key** is smaller than the middle element, then the **left** side is used for next search.
 - If the **key** is larger than the middle element, then the **right** side is used for next search.
- This process is continued until the **key** is found or the total search space is exhausted.

Binary Search Algorithm

- Consider array `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`, & `target = 23`.

Initially

Find Key = 23 using Binary Search

	0	1	2	3	4	5	6	7	8	9
<code>arr[] =</code>	2	5	8	12	16	23	38	56	72	91

Step 1

Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.

				Key						
				23						
	0	1	2	3	4	5	6	7	8	9
<code>arr[] =</code>	2	5	8	12	16	23	38	56	72	91
	Low = 0			Mid = 4		High = 9				

Binary Search Algorithm

- Consider array `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`, & `target = 23`.

Initially

Find Key = 23 using Binary Search

	0	1	2	3	4	5	6	7	8	9
<code>arr[] =</code>	2	5	8	12	16	23	38	56	72	91

Step 1

Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.

	0	1	2	3	4	5	6	7	8	9
<code>arr[] =</code>	2	5	8	12	16	23	38	56	72	91
	Low = 0				Mid = 4	High = 9				

Binary Search Algorithm

- It can be implemented in the following two ways
 - Iterative Algorithm
 - Recursive Algorithm

Binary Search Algorithm

```
// An iterative binary search function.
int binarySearch(int arr[], int low, int high, int x)
{
    while (low <= high) {
        int mid = low + (high - low) / 2;
        // Check if x is present at mid
        if (arr[mid] == x)
            return mid;
        // If x greater, ignore left half
        if (arr[mid] < x)
            low = mid + 1;
        // If x is smaller, ignore right half
        else
            high = mid - 1;
    }
    // element was not present
    return -1;
}
```

```
// Driver code
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;
    int result=binarySearch(arr, 0, n - 1, x);
    (result == -1) ? printf("Element is not
                        present in array") : printf
                        ("Element is present at index %d",
                        result);

    return 0;
}
```

Binary Search Algorithm

```
int binarySearch(int arr[], int low, int high, int x)
{
    if (high >= low) {
        int mid = low + (high - low) / 2;
        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;
        // If element is smaller than mid, then
        // it can only be present in left subarray
        if (arr[mid] > x)
            return binarySearch(arr, low, mid - 1, x);
        // Else the element can only be present
        // in right subarray
        return binarySearch(arr, mid + 1, high, x);
    }
}
```

```
// element is not present in array
return -1;
}
// Driver code
int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int x = 10;
    int result=binarySearch(arr, 0, n - 1, x);
    (result == -1) ? printf("Element is not
                        present in array") : printf
                        ("Element is present at index %d",
                        result);

    return 0;
}
```

Binary Search Algorithm

- Consider array `arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`, & `target = 23`.

Step 2

Key is less than the current mid 56.
The search space moves to the left.



Step 3

If the key matches the value of the mid element,
the element is found and stop search.



Binary Search Algorithm

- **Time Complexity:**

- Best Case: $O(1)$

- Average Case: $O(\log N)$

- Worst Case: $O(\log N)$

- **Auxiliary Space:** $O(1)$, If the recursive call stack is considered then the auxiliary space will be $O(\log N)$.

Applications of Binary Search Algorithm:

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- It can be used for searching in computer graphics such as algorithms for ray tracing or texture mapping.
- It can be used for searching a database.

Binary Search Algorithm

Advantages:

- Binary search is faster than linear search, especially for large arrays.
- More efficient than other searching algorithms with a similar time complexity, such as interpolation search or exponential search.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.

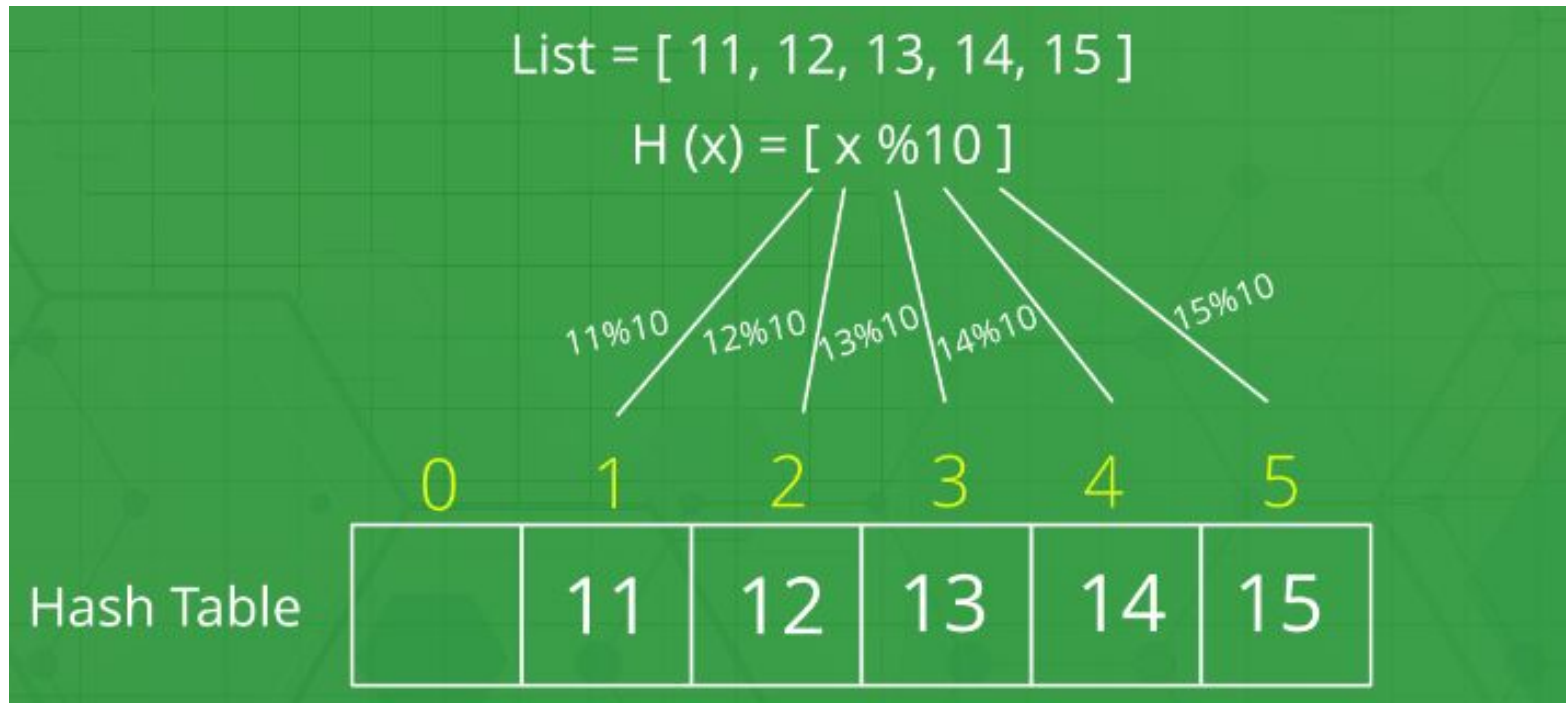
Disadvantages:

- The array should be sorted.
- Binary search requires that the data structure being searched be stored in contiguous memory locations.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered

Hashing

- **Hash Table- Concepts-hash table, hash function,**
- **basic operations, bucket, collision, probe, synonym, overflow,**
- **open hashing, closed hashing, perfect hash function, load density, full table, load factor, rehashing,**
- **issues in hashing, hash functions- properties of good hash function,**
- **division, multiplication, extraction, mid-square, folding and universal,**
- **Collision resolution strategies- open addressing and chaining,**
- **Hash table overflow- open addressing and chaining,**
- **Extendible hashing.**

Hashing



What is Hashing

- Hashing is the technique used for **storing and retrieving information** as fast as possible. It is based on hash table.
- **Why hashing ?** It is used to **perform optimal search.**

Searching	Best case	Average case	Worst case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Hashing	$O(1)$	$O(1)$	$O(n)$

Hashing

Need for Hash data structure

- Every day, the data on the internet is increasing multifold and it is always a **struggle to store this data efficiently**. In day-to-day programming, this **amount of data** might **not** be that **big**, but still, it **needs** to be **stored**, **accessed**, and **processed easily** and **efficiently**. A very common data structure that is used for such a purpose is the **Array data structure**.
- Array was already there, **what was the need for a new data structure!** The answer □ is in the word “**efficiency**“. Though storing in **Array** takes **$O(1)$ time**, **searching** in it takes **at least $O(\log n)$ time**. This time appears to be **small**, but **for a large data set**, it can cause a **lot of problems** and this, in turn, makes the **Array data structure inefficient**.
- So now **we are looking for a data structure** that can **store** the **data** and **search** in it in **constant time**, i.e. in **$O(1)$ time**. This is how **Hashing data structure** came into play.

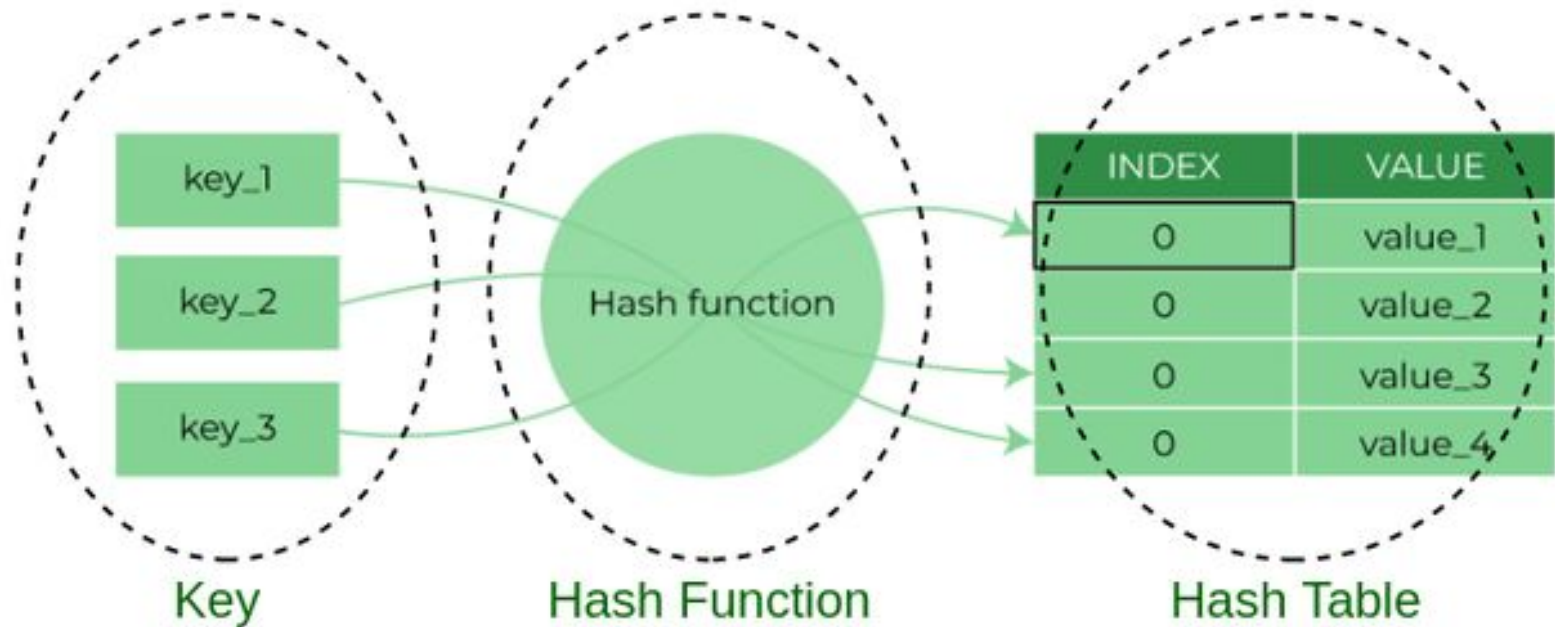
Hashing

Components of Hashing

There are majorly three components of hashing:

- **Key:** A **Key** can be anything **string** or **integer** which is fed as **input** in the hash function the technique that **determines** an **index** or **location** for **storage** of an **item** in a data structure.
- **Hash Function:** The **hash function** receives the **input key** and **returns** the **index** of an element in an **array** called a **hash table**. The index is known as the **hash index**.
- **Hash Table:** Hash table is a data structure that **maps keys to values** using a special function called a **hash function**. Hash stores the data in an **associative** manner in an **array** where **each data** value has its **own unique index**.

Hashing



Components of Hashing

Hashing

How does Hashing work?

- Suppose - set of strings {"ab", "cd", "efg"} and like to store it in a table.
- Main **objective** here is to **search or update** the values stored in the table quickly in **$O(1)$ time** and **ordering** of strings in the table is **not concerned**. So the given **set of strings** can **act** as a **key** and the string itself will act as the value of the string but how to store the value corresponding to the key?
- **Step 1:** hash functions (which is some mathematical formula) are used to calculate the hash value which acts as the index of the data structure where the value will be stored.
- **Step 2:** So, let's assign
 - "a" = 1,
 - "b"=2, .. etc, to all alphabetical characters.

Hashing

- **Step 3:** Therefore, the numerical value by **summation of all characters** of the string:
 - “ab” = $1 + 2 = 3$,
 - “cd” = $3 + 4 = 7$,
 - “efg” = $5 + 6 + 7 = 18$
- **Step 4:** Now, assume that table of **size 7** to store these strings. The **hash function** that is used here is the **sum of the characters** in **key mod Table size**. Compute the location of the string in the array by taking the **sum(string) mod 7**.
- **Step 5:** So we will then store
 - “ab” in $3 \bmod 7 = 3$,
 - “cd” in $7 \bmod 7 = 0$, and
 - “efg” in $18 \bmod 7 = 4$.

Hashing

0	1	2	3	4	5	6
cd			ab	efg		

- The technique **enables** us to **calculate** the **location** of a given string by using a **simple hash function** and **rapidly find** the value that is stored in that location.
- Therefore the idea of hashing seems like a great way to store **(key, value)** pairs of the data in a table.

Hash function

What is a Hash function?

- The hash **function** creates a **mapping between key and value**, this is done through the use of **mathematical formulas** known as **hash functions**.
- The **result** of the **hash function** is referred to as a **hash value** or **hash**. The **hash value** is a **representation** of the **original string** of characters but **usually smaller** than the **original**.
- For example: Consider an **array** as a **Map** where the **key** is the **index** and the **value** is the **value** at that **index**. So for an array **A** if we have **index i** which will be treated as the key then we can **find** the **value** by simply looking at the **value at A[i]**.

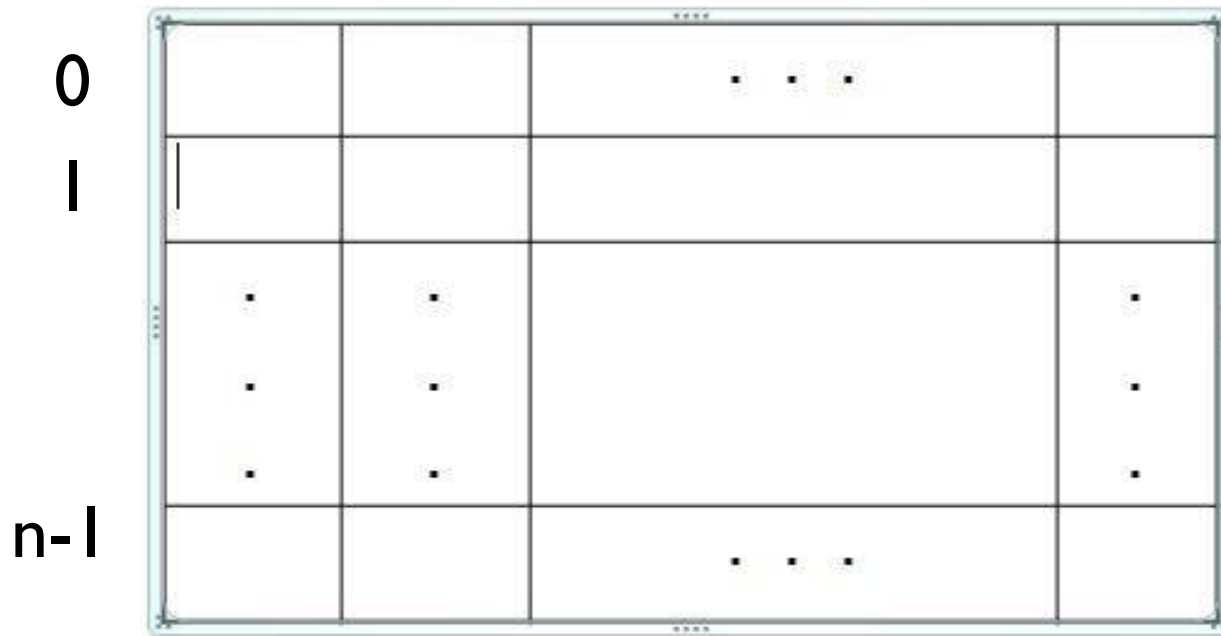
Hash Table

- Hash table is the data structure, supports **constant time insertion, deletion** and **search on average**.
- Key-value pairs are stored in *hash table*.
 - A hash table is partitioned into many ***buckets***.
 - Each bucket has many ***slots***.
 - Each slot holds one **record**.
 - A **hash function** $H(K)$ transforms the **identifier** (key) into an **address** in the **hash table**

Hash Table

- Number of slots m and n buckets

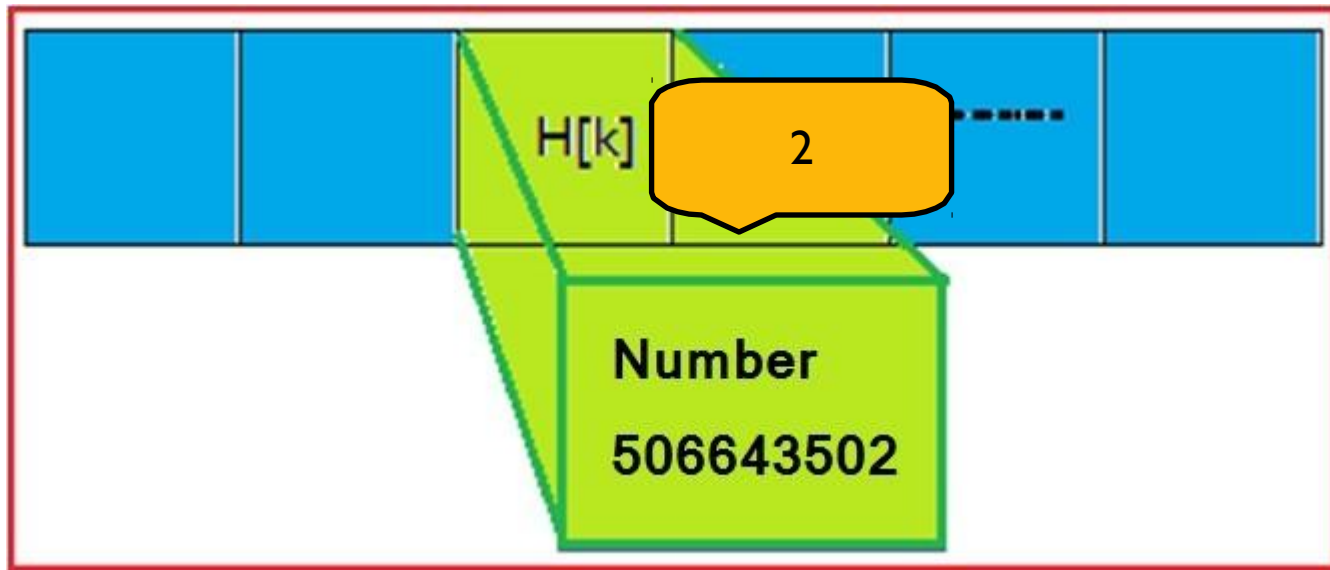
- 0 1 $m-1$



Hash Table

- The simplest kind of hash table is an array of records. Each record has a special field called key.

- [0] [1] [2] [3] [499]



Good Hash function - features

Characteristics of good hash function

- Minimum collision - A good hash function is one which distribute keys evenly among the slots in hash table, easy and quick to compute, minimize collision.
- High gain factor (distributes keys evenly). Like say we have 10 locations in the hash table, then almost 9 locations should have keys. It should not be the case that 4-5 locations have keys only where the keys collided.
- Have a high load factor. Load factor means the number of keys stored/hash table size
- Easy to compute

Perfect Hash function

- Hash function should be simple to computer.
- Number of collision should be less
- The hash function uses all the input data.
- The hash function "uniformly" distributes the data across the entire set of possible hash values.
- The hash function generates very different hash values for similar strings.

Advantages :

- A perfect hash function with values in a limited range can be used for efficient lookup operations.
- No need to apply collision resolution techniques

Hashing

- **Load factor** is defined as (m/n) where n is the total size of the hash table and m is the preferred number of entries that can be inserted before an increment in the size of the underlying data structure is required.

$$\text{Load Factor } (\alpha) = \frac{\text{Number of elements present in the hash table}}{\text{Total size of the hash table}}$$

If we have the initial capacity of HashTable = 16.

We insert the first element and now check if we need to increase the size of the HashTable capacity or not.

It can be determined by the formula:

Load factor and Load density

- Load Factor: The ratio of the number of items in a table to the table's size is called the load factor.

Load Density :

- The identifier density of a hash table is the ratio n/T , where n is the number of identifiers in the table.
 T is total number of possible element.
- The loading density or loading factor of a hash table is $\alpha = n / (sb)$.

Load factor and Load density

Load Density :

- Consider the hash table with $b = 26$ buckets and $s = 2$.
We have $n = 10$ distinct identifiers, each representing a C library function.
- This table has a loading factor, α , of
$$\alpha = n / (sb) = 10/52 = 0.19$$

Hash function –Basic Operations

- The primary operations of a hash table are **search, insert, and delete**.
- The search operation is used to locate the element within a hash table.
- The insert operation is used to add elements into the hash table.
- The delete operation removes elements from the hash table.

Hash function –Issues:

- Table size
- Hash function
- Collision resolution

Hashing

Size of hashmap (m) / number of buckets (n)

In this case, the size of the hashmap is 1, and the bucket size is 16. So,
 $1/16=0.0625$. Now compare this value with the default load factor.

$0.0625 < 0.75$ So, no need to increase the hashmap size.

We do not need to increase the size of hashmap up to 12th element, because

$12/16=0.75$ As soon as we insert the 13th element in the hashmap, the size of hashmap is increased because:

$13/16=0.8125$ Which is greater than the default hashmap size.

$0.8125 > 0.75$ Now we need to increase the hashmap size

Hashing Function– Types/Methods

- **Hash Function**
 - **Division**
 - **Mid-square hashing**
 - **Folding**
 - **Multiplication**
- **Collision resolution Techniques**
 - **Direct chaining (Open Hashing)**
 - **Open Addressing**
 - **Linear Probing**
 - **Quadratic Probing**
 - **Double Hashing**
- **Applications of hashing**

Hash table

- In order to insert a new record, the key is converted to an array index.
- The index is called the hash value of the key.
- Commonly used method to find hash value is $(\text{key}) \bmod (\text{size of hash table})$
$$= 506643502 \bmod 500 = 2$$

store at index 2

Division hash method

- The key K is divided by hash table size m and the remainder is used as the hash address of key.
- $H(K) = K \bmod m$
- This gives the indexes in the range 0 to $m-1$.
- Ex –if key = 58 , $m=10$
- $H(k) = 58 \bmod 10 = 8$

Mid- square Method

- The key K is multiplied by itself and the address is obtained by selecting appropriate number of digits from the middle of the square.
- The number of digits selected depends on the size of the table.
- Ex – if key = 123456
- $(123456)^2 = 15241383936$

Folding Method

- The key is partitioned into a number of parts, each of which has the same length as the required address with the possible exception of the last part.
- The parts are then added together, ignoring the final carry, to form an address.
- Ex- if key = 356942781 into three digit address.
- $p_1 = 356$, $p_2 = 942$, $p_3 = 781$ are added
- $356 + 942 + 781 = 2079$

Multiplication Method

This method involves the following steps:

- ❑ Choose a constant value A such that $0 < A < 1$.
- ❑ Multiply the key value with A .
- ❑ Extract the fractional part of kA .
- ❑ Multiply the result of the above step by the size of the hash table i.e. M .
- ❑ The resulting hash value is obtained by taking the floor of the result obtained in step 4.

$$h(K) = \text{floor}(M (kA \bmod 1))$$

M is the size of the hash table.

k is the key value.

A is a constant value.

Multiplication Method

$$k = 12345$$

$$A = 0.357840$$

$$M = 100$$

$$\begin{aligned} h(12345) &= \text{floor}[100 (12345 * 0.357840 \bmod 1)] \\ &= \text{floor}[100 (4417.5348 \bmod 1)] \\ &= \text{floor}[100 (0.5348)] \\ &= \text{floor}[53.48] \\ &= 53 \end{aligned}$$

Multiplication Method

Pros:

The advantage of the multiplication method is that it can work with any value between 0 and 1, although there are some values that tend to give better results than the rest.

Cons:

The multiplication method is generally suitable when the table size is the power of two, then the whole process of computing the index by the key using multiplication hashing is very fast.

Commonly used hash functions:

1. SHA (Secure Hash Algorithm): SHA is a family of cryptographic hash functions designed by the National Security Agency (NSA) in the United States. The most widely used SHA algorithms are SHA-1, SHA-2, and SHA-3. Here's a brief overview of each:

SHA-1: SHA-1 is a 160-bit hash function that was widely used for digital signatures and other applications. However, it is no longer considered secure due to known vulnerabilities.

SHA-2: SHA-2 is a family of hash functions that includes SHA-224, SHA-256, SHA-384, and SHA-512. These functions produce hash values of 224, 256, 384, and 512 bits, respectively. SHA-2 is widely used in security protocols such as SSL/TLS and is considered secure.

SHA-3: SHA-3 is the latest member of the SHA family and was selected as the winner of the NIST hash function competition in 2012. It is designed to be faster and more secure than SHA-2 and produces hash values of 224, 256, 384, and 512 bits.

Commonly used hash functions:

2. CRC (Cyclic Redundancy Check): CRC is a non-cryptographic hash function used primarily for error detection in data transmission. It is fast and efficient but is not suitable for security purposes. The basic idea behind CRC is to append a fixed-length check value, or checksum, to the end of a message. This checksum is calculated based on the contents of the message using a mathematical algorithm, and is then transmitted along with the message.

Commonly used hash functions:

3. MD5 (Message Digest 5): MD5 is a widely-used cryptographic hash function that produces a 128-bit hash value. It is fast and efficient but is no longer recommended for security purposes due to known vulnerabilities. The basic idea behind MD5 is to take an input message of any length, and produce a fixed-length output, known as the hash value or message digest. This hash value is unique to the input message, and is generated using a mathematical algorithm that involves a series of logical operations, such as bitwise operations, modular arithmetic, and logical functions. MD5 is widely used in a variety of applications, including digital signatures, password storage, and data integrity checks. However, it has been shown to have weaknesses that make it vulnerable to attacks. In particular, it is possible to generate two different messages with the same MD5 hash value, a vulnerability known as a collision attack.

Collision

What is a Hash Collision?

- ❑ A hash collision occurs when two different keys map to the same index in a hash table. This can happen even with a good hash function, especially if the hash table is full or the keys are similar.

Causes of Hash Collisions:

- ❑ **Poor Hash Function:** A hash function that does not distribute keys evenly across the hash table can lead to more collisions.
- ❑ **High Load Factor:** A high load factor (ratio of keys to hash table size) increases the probability of collisions.
- ❑ **Similar Keys:** Keys that are similar in value or structure are more likely to collide.

Collision

When the two different values have the same value, then the problem occurs between the two values, known as a collision. the value is stored at index 6. If the key value is 26, then the index would be:

$$h(26) = 26 \% 10 = 6$$

$$H(36) = 36 \% 10 = 6$$

Therefore, two values are stored at the same index, i.e., 6, and this leads to the collision problem.

To resolve these collisions, we have some techniques known as collision techniques.

Collision

Problems with hashing

We want a **perfect hash function**, one that maps each unique key to a unique index. This is not realistic but we can often come close enough.

It is likely that **collisions** occur, where two or more unique keys map (hash) to the same index.

These colliding two keys are called **synonyms**

Probing involves **finding another location in the hash table for the data element when a collision occurs**. The simplest form of probing is linear probing, where the hash table is searched sequentially to find an empty location.

Example

- Find the number of collision when the following keys are reduced to hash table. Hash function is $\text{key mod } n$ and hash table size is 7.

Keys are : 10, 100, 33, 45, 58, 2

$10 \bmod 7$	3
$100 \bmod 7$	2
$33 \bmod 7$	5
$45 \bmod 7$	3 (1)
$58 \bmod 7$	2 (2)
$2 \bmod 7$	2 (3)

Collision resolution Techniques

- **Collision resolution Techniques**

- **Direct chaining (Open Hashing)**
- **Open Addressing**
 - **Linear Probing**
 - **Quadratic Probing**
 - **Double Hashing**

Collision Resolution techniques:

- When an element is inserted, it hashes to the same value as an already inserted element, then we have a collision and need to resolve it.
- There are several collision resolution techniques:

-

▪ **Direct/Separate chaining (Open hashing)**

Create an array of linked list of words, so that the item can be inserted into the linked list if collision occurs.

-

▪ **Open addressing (closed hashing)**

Search the array in some systematic way for an empty cell and insert the new item there if collision occurs.

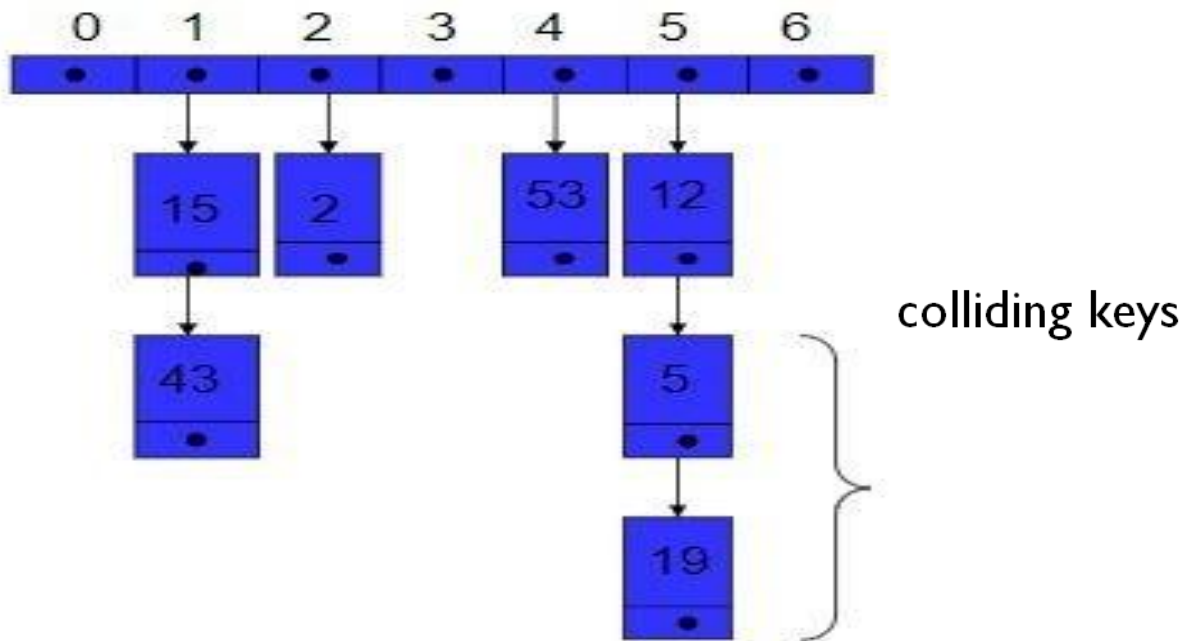
- Linear Probing
- Quadratic Probing
- Double Hashing

Direct /Separate Chaining

keep a linked list of all the **synonyms**

- Keys are stored as an array of linked lists.

-



- Hash Function is $\text{key} \% 7$.
- Insert pairs whose keys are 15, 2, 12, 53, 5, 43, 19

Linear Probing

- Search sequentially for vacant cells, incrementing the index until an empty cell is found.

- $H_i(X) = (\text{Hash}(K) + i) \% \text{TableSize}$

$$H_0(K) = \text{Hash}(K) \% \text{TableSize}$$

$$H_1(K) = \text{Hash}(K) + 1 \% \text{TableSize}$$

$$H_2(K) = \text{Hash}(K) + 2 \% \text{TableSize}$$

$$H_3(K) = \text{Hash}(K) + 3 \% \text{TableSize}$$

0				4					8				12
26	14	52				6	46	33				37	24

- Insert pairs whose keys are 26, 14, 52, 6, 37, 24, 46, 33

- Insert 12, 01, 04, 03, 07, 08, 10, 02, 05, 14

[illegible]

- Insert 12, 01, 04, 03, 07, 08, 10, 02, 05, 14

Table MOD as hash function and linear probing without replacement

[illegible]

Linear Probing

EXAMPLE Suppose $\text{Max} = 8$ and keys A, B, C, D have hash values $\text{Hash}(A) = 3$, $\text{Hash}(B) = 0$, $\text{Hash}(C) = 4$, and $\text{Hash}(D) = 3$. Use linear probing for these values.

Solution Linear probing is the simplest strategy where $\text{HashQ}(\text{Key}) = (\text{HashQ}(\text{Key}) + i) \text{ MOD Max}$.

Suppose we wish to insert D and find that bucket 3 has been filled already, then we would try buckets 4, 5, 6, 7, 0, 1, and 2 in sequence. We find bucket 5 empty and we store D .

0	B
1	
2	
3	A
4	C
5	D
6	
7	

Linear Probing

A disadvantage to linear probing is the tendency for **clustering**; items become clustered in the table. This means that if many collisions occur at the same hash value, a number of surrounding slots will be filled by the linear probing resolution.

This will have an impact on other items that are being inserted, as we saw when we tried to add the item 20 above. A cluster of values hashing to 0 had to be skipped to finally find an open position.

0	1	2	3	4	5	6	7	8	9	10
77	44	55	20	26	93	17	None	None	31	54

Figure : A Cluster of Items for Slot 0

Quadratic Probing

- The step is the square Of step number

- $H_i(K) = (\text{Hash}(K) \pm i^2) \% \text{TableSize}$

$$H_0(K) = \text{Hash}(K) \% \text{TableSize}$$

$$H_1(K) = \text{Hash}(K) + 1 \% \text{TableSize}$$

$$H_2(K) = \text{Hash}(K) + 4 \% \text{TableSize}$$

$$H_3(K) = \text{Hash}(K) + 9 \% \text{TableSize}$$

0				4				8					12
26	14			52		6	46					37	24

Insert pairs whose keys are 26, 14, 52, 6, 37, 24, 46, 32

Quadratic Probing

Insert pairs whose keys are 7, 22, 17, 32, 5, 24 . Tablesize or Max =7

Table Keys and quadratic probing

Index	Key		Index	Key
0	7		0	7
1	22		1	22
2		insert 24 →	2	24
3	17		3	17
4	32		4	32
5	5		5	5
6			6	

Quadratic Probing

Insert pairs whose keys are 7, 22, 17, 32, 5, 24 . Tablesize or Max =7

while inserting 24, the address we get is

$$\begin{aligned} & [\text{Hash}(24) + (1)^2 \text{ MOD } 7] \\ &= ((24 \text{ MOD } 7) + 1) \text{ MOD } 7 \\ &= 4 \end{aligned}$$

location 4 is already occupied. So using Quadratic Probing

$$\begin{aligned} & [\text{Hash}(24) - (1)^2 \text{ MOD } 7] \\ &= (24 \text{ MOD } 7) - 1 \text{ MOD } 7 \\ &= 2 \end{aligned}$$

Double Hashing

- Hash the key a second time using a different hash function and use the result as the step size.
- $H_i(K) = (\text{Hash}_1(K) + i * \text{Hash}_2(K)) \bmod \text{TableSize}$
for $i = 0, 1, 2, \dots$
- Let $\text{Hash}_1(K) = K \bmod 13 \quad \dots \text{TableSize} = 13$
- $\text{Hash}_2(K) = R - (K \bmod R)$
 $R = \text{small prime number than table size } 13 = 7$
- $H_0(K) = ((K \bmod 13) + 0 * (7 - (K \bmod 7))) \bmod 13$

Double hashing

- Insert pairs whose keys are 26, 14, 52, 6, 37, 24, 46, 32
- To insert 26 , $h1 = 26 \% 13 = 0$, $h2 = 7 - (26 \% 7) = 2$
- $H(26) = (h1 + 0 * h2) \% 13 = (0 + 0 * 2) \% 13 = 0$ (index =0)
- To insert 14 , $h1 = 14 \% 13 = 1$, $h2 = 7 - (14 \% 7) = 7$
- $H(14) = (h1 + 0 * h2) \% 13 = (1 + 0 * 7) \% 13 = 1$ (index =1)
- To insert 52 , $h1 = 52 \% 13 = 0$, $h2 = 7 - (52 \% 7) = 4$
- $H(52) = (h1 + 0 * h2) \% 13 = (0 + 0 * 4) \% 13 = 0$ (collision)
 $H_1(52) = (h1 + 1 * h2) \% 13 = (0 + 1 * 4) \% 13 = 4$ (index =4)
- To insert 6 , $h1 = 6 \% 13 = 6$, $h2 = 7 - (6 \% 7) = 1$
- $H(6) = (h1 + 0 * h2) \% 13 = (6 + 0 * 1) \% 13 = 6$ (index = 6)

0	4				8				12			
26	14	24		52		6			32	46	37	

Double hashing

Insert pairs whose keys are 12,01,18,56,79,49. Tablesize or Max =10 & Hashing function is $\text{key} \% 10$. Perform Double hashing

Table Double hashing

	Initially empty	Insert 12	Insert 01	Insert 18	Insert 56	Insert 79	Insert 49
0							
1			01	01	01	01	01
2		12	12	12	12	12	12
3							49
4							
5							49
6					56	56	56
7							
8				18	18	18	18
9						79	79

Rehashing

Rehashing is a technique in which the table is resized, i.e., the size of table is doubled by creating a new table. It is preferable if the total size of table is a prime number. There are situations in which the rehashing is required.

- When table is completely full
- With quadratic probing when the table is filled half.
- When insertions fail due to overflow.

In such situations, we have to transfer entries from old table to the new table by re computing their positions using hash functions

Rehashing

Insert pairs whose keys are 7,13,15,73,74. Tablesize or Max =7
& Hashing function is $\text{key} \% 7$. Perform Rehashing

Table	Rehashing	Table	New table when hash table (Table 11.14) is 70% full
Insert 7, 15, 13, 74, 73			
0	7	0	
1	15	1	
2		2	
3	73	3	
4	74	4	
5		5	73
6	13	6	74
		7	7
		8	
		9	
		10	
		11	
		12	
		13	13
		14	
		15	15
		16	

Rehashing

Why rehashing?

Rehashing is done because whenever key value pairs are inserted into the map, the load factor increases, which implies that the time complexity also increases as explained above.

This might not give the required time complexity of $O(1)$. Hence, rehash must be done, increasing the size of the bucketArray so as to reduce the load factor and the time complexity

Chaining vs. Open Addressing

- **Chaining:** uses extra memory for links and search time is $O(n)$
- **Open addressing :** needs extra memory within the table to terminate the probe sequence. Search time is $O(n)$ and deletion of data from hash table is difficult

Chaining vs. ReHashing

Table Comparison of chaining and rehashing

Chaining	Rehashing
Unlimited number of synonyms can be handled.	A limited but good number of synonyms are taken care of.
Additional cost to be paid is an overhead of multiple linked lists.	The table size is doubled but <u>no additional fields of links are to be maintained.</u>
Sequential search through the chain takes more time.	Searching is faster when compared to chaining.

Hash Table overflow- handled by Open Addressing

Overflow

- The result of more keys hashing to the same address and if there is no room in the bucket, then it is said that overflow has occurred
- Collision and overflow are synonymous when the bucket is of size 1

When linear open addressing is used to handle overflows, a hash table search for an identifier I proceeds as follows:

1. Compute $\text{Hash}(I)$

2. Examine identifiers position

$\text{Table}[\text{Hash}(I)], \text{Table}[\text{Hash}(I) + 1], \dots, \text{Table}[\text{Hash}(I) + i]$, in order until:

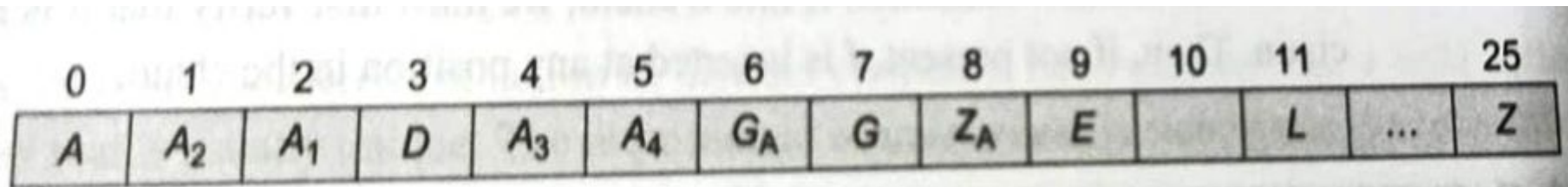
(a) If $\text{Table}[\text{Hash}(I) + j] = I$ then

In this case I is found.

(b) If $\text{Table}[\text{Hash}(I) + j]$ is NULL, then I is not in the table.

(c) If we return to the start position $\text{Hash}(I)$, then the table is full and I is not in the table.

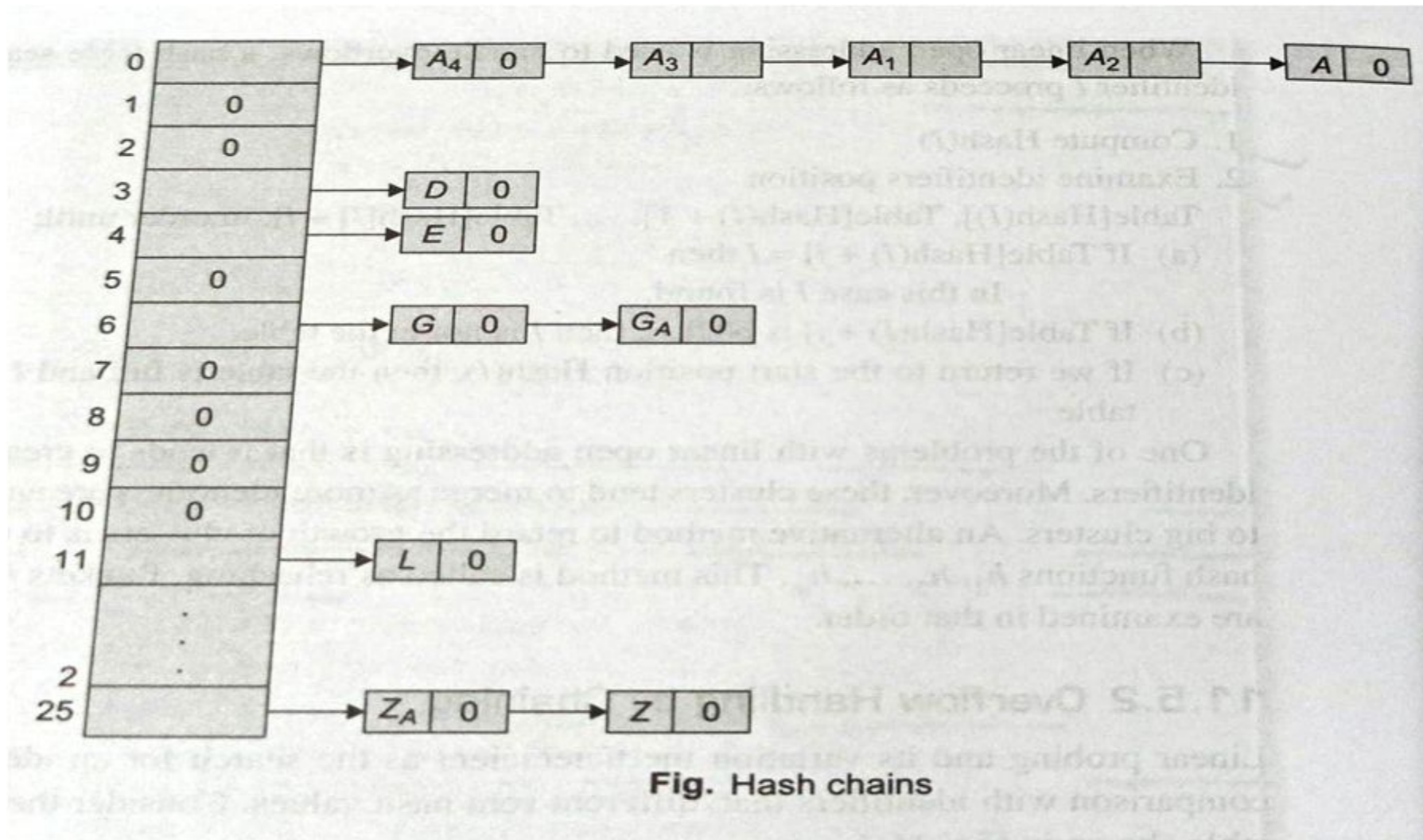
Hash table Overflow- handled by Chaining



0	1	2	3	4	5	6	7	8	9	10	11	...	25
A	A ₂	A ₁	D	A ₃	A ₄	G _A	G	Z _A	E		L	...	Z

Fig. Chaining

Hash table Overflow- handled by Chaining



Extendible Hashing

- ❑ It is a **dynamic hashing** method wherein directories, and buckets are used to hash data.
- ❑ It is an aggressively **flexible** method in which the hash function also experiences dynamic changes.

Extendible Hashing

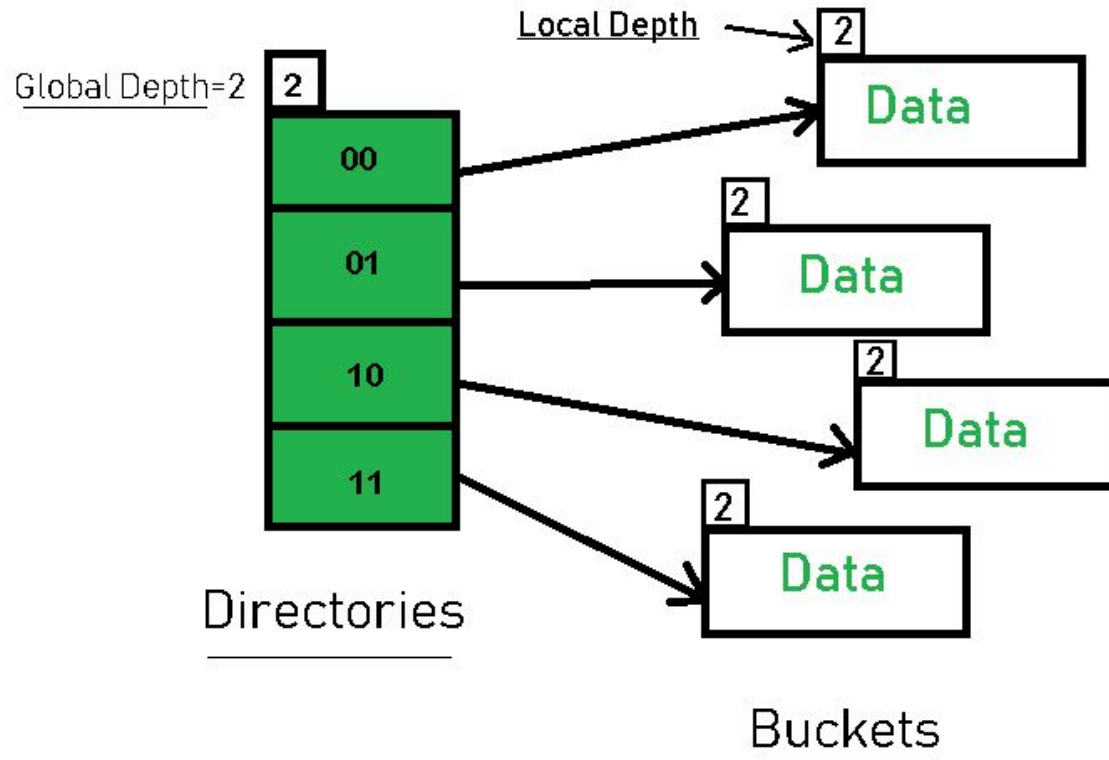
- ❑ **Directories:** These containers **store pointers** to buckets. Each directory is given a **unique id** which may change each time when expansion takes place. The **hash function returns this directory id** which is used to **navigate** to the appropriate **bucket**. Number of **Directories = $2^{\text{Global Depth}}$** .
- ❑ **Buckets:** They **store the hashed keys**. Directories point to buckets. A bucket may **contain more than one pointers** to it if its **local depth is less than the global depth**.
- ❑ **Global Depth:** It is **associated with the Directories**. They denote the **number of bits** which are **used** by the **hash function** to categorize the keys.

Global Depth = Number of bits in directory id.

Extendible Hashing

- ❑ **Local Depth:** It is the **same** as that of **Global Depth except** for the fact that **Local Depth is associated with the buckets** and **not the directories**. Local depth in accordance with the global depth is **used to decide** the **action** that to be performed in case an **overflow occurs**. Local Depth is **always less than or equal to** the Global Depth.
- ❑ **Bucket Splitting:** When the number of elements in a bucket **exceeds a particular size**, then the bucket is **split** into **two parts**.
- ❑ **Directory Expansion:** Directory Expansion Takes place **when** a **bucket overflows**. Directory Expansion is performed **when** the local depth of the **overflowing bucket is equal to** the global depth.

Extendible Hashing

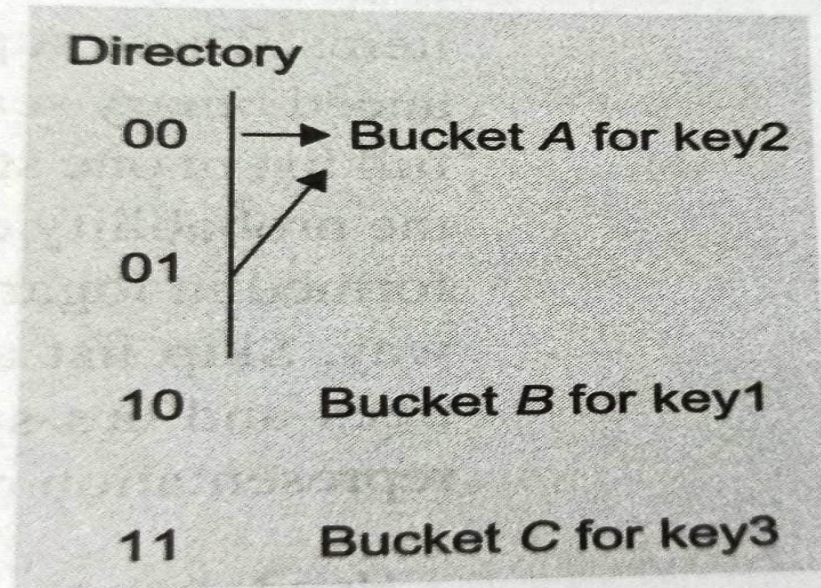
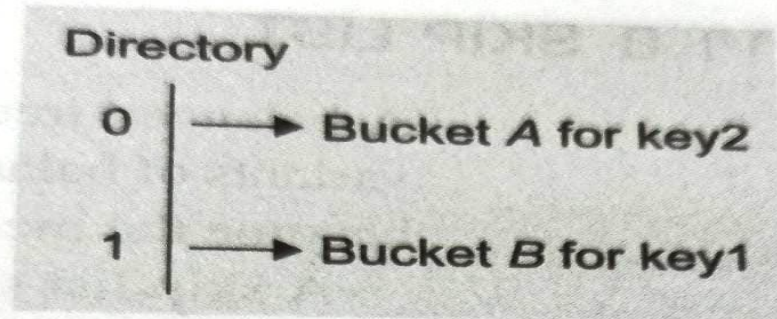


Extendible Hashing

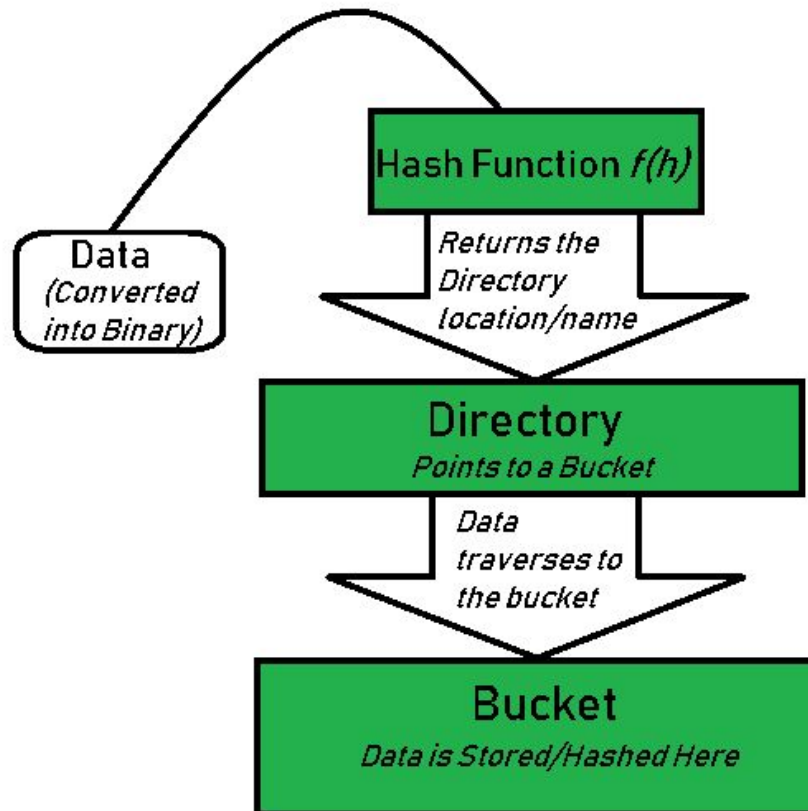
Extendible Hashing

The keys to be used are

1. $h(\text{key1}) = 100101$
2. $h(\text{key2}) = 011110$
3. $h(\text{key3}) = 110110$



Extendible Hashing



Extendible Hashing

- ❑ **Step 1 – Analyze Data Elements:** Data elements may exist in various forms eg. Integer, String, Float, etc.. Currently, let us consider data elements of type integer. eg: 49.
- ❑ **Step 2 – Convert into binary format:** Convert the data element in Binary form. For string elements, consider the ASCII equivalent integer of the starting character and then convert the integer into binary form. Since we have 49 as our data element, its binary form is 110001.
- ❑ **Step 3 – Check Global Depth of the directory.** Suppose the global depth of the Hash-directory is 3.

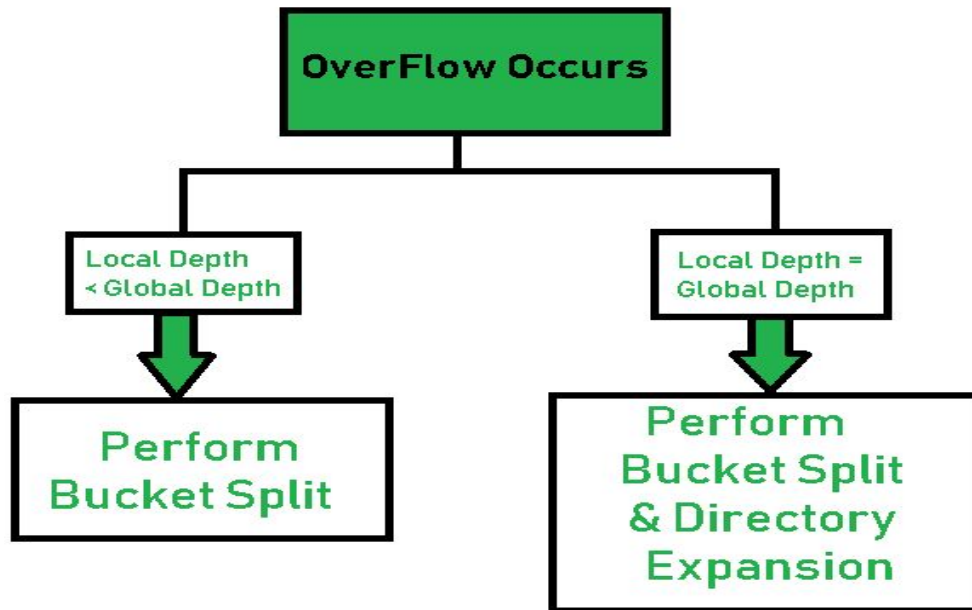
Extendible Hashing

- ❑ **Step 4 – Identify the Directory:** Consider the ‘Global-Depth’ number of LSBs in the binary number and match it to the directory id.
Eg. The binary obtained is: 1 10001 and the global-depth is 3. So, the hash function will return 3 LSBs of 110001 viz. 001.
- ❑ **Step 5 – Navigation:** Now, navigate to the bucket pointed by the directory with directory-id 001.
- ❑ **Step 6 – Insertion and Overflow Check:** Insert the element and check if the bucket overflows. If an overflow is encountered, go to **step 7** followed by **Step 8**, otherwise, go to **step 9**

Extendible Hashing

- ❑ **Step 7 – Tackling Over Flow Condition during Data Insertion:** Many times, while inserting data in the buckets, it might happen that the Bucket overflows. In such cases, we need to follow an appropriate procedure to avoid mishandling of data.
First, Check if the local depth is less than or equal to the global depth. Then choose one of the cases below.
 - ❑ **Case1:** If the local depth of the overflowing Bucket is equal to the global depth, then Directory Expansion, as well as Bucket Split, needs to be performed. Then increment the global depth and the local depth value by 1. And, assign appropriate pointers.
Directory expansion will double the number of directories present in the hash structure.
 - ❑ **Case2:** In case the local depth is less than the global depth, then only Bucket Split takes place. Then increment only the local depth value by 1. And, assign appropriate pointers.

Extendible Hashing



Step 8 – Rehashing of Split Bucket Elements: The Elements present in the overflowing bucket that is split are rehashed w.r.t the new global depth of the directory.

Step 9 – The element is successfully hashed.

Extendible Hashing

Example based on Extendible Hashing: Now, let us consider a prominent example of hashing the following elements:

16,4,6,22,24,10,31,7,9,20,26.

Bucket Size: 3 (Assume)

Hash Function: Suppose the global depth is X . Then the Hash Function returns X LSBs.

Solution: First, calculate the binary forms of each of the given numbers.

Extendible Hashing

16- 10000

4- 00100

6- 00110

22- 10110

24- 11000

10- 01010

31- 11111

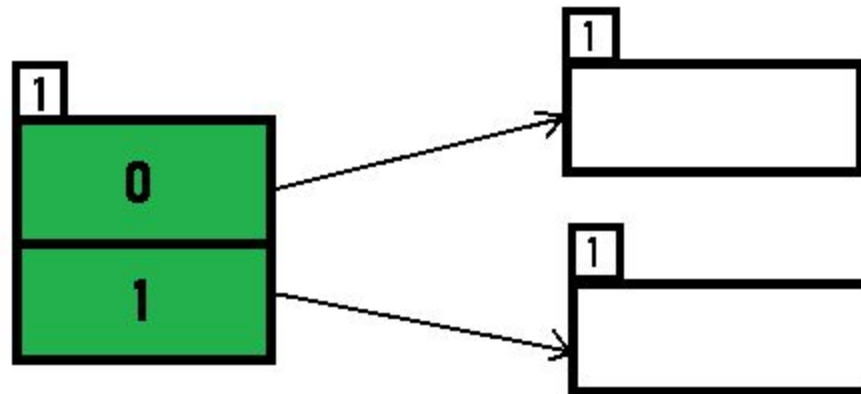
7- 00111

9- 01001

20- 10100

26- 11010

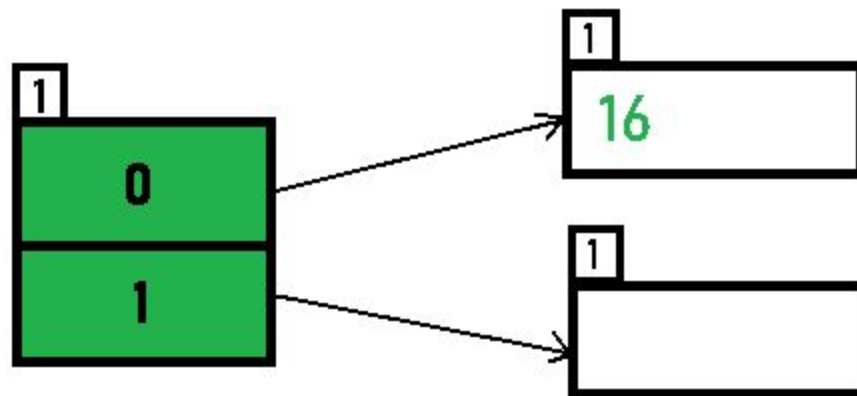
Initially, the global-depth and local-depth is always 1.



Extendible Hashing

Inserting 16:

The binary format of 16 is 10000 and global-depth is 1. The hash function returns 1 LSB of 10000 which is 0. Hence, 16 is mapped to the directory with id=0.

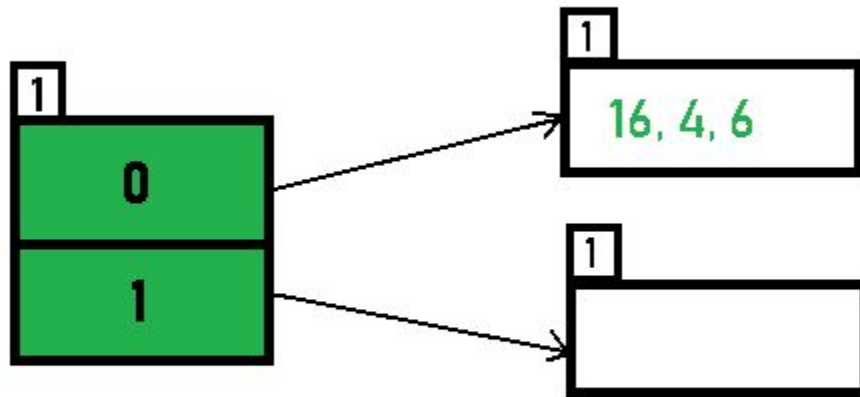


$Hash(16) = 1000\underline{0}$

Extendible Hashing

Inserting 4 and 6:

Both 4(100) and 6(110) have 0 in their LSB.

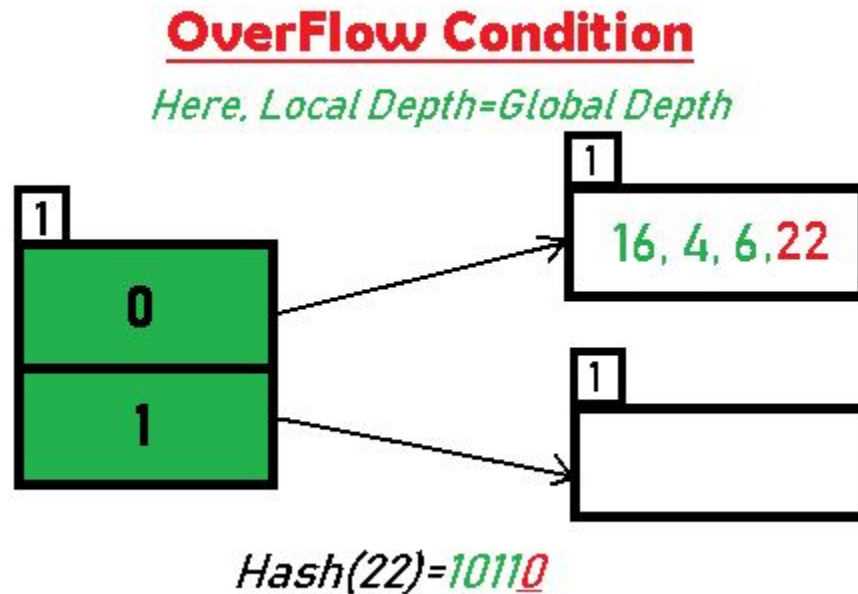


$Hash(4) = 100$

$Hash(6) = 110$

Extendible Hashing

Inserting 22: The binary form of 22 is 10110. Its LSB is 0. The bucket pointed by directory 0 is already full. Hence, Over Flow occurs.



Extendible Hashing

As directed by **Step 7-Case 1**, Since Local Depth = Global Depth, the bucket splits and directory expansion takes place.

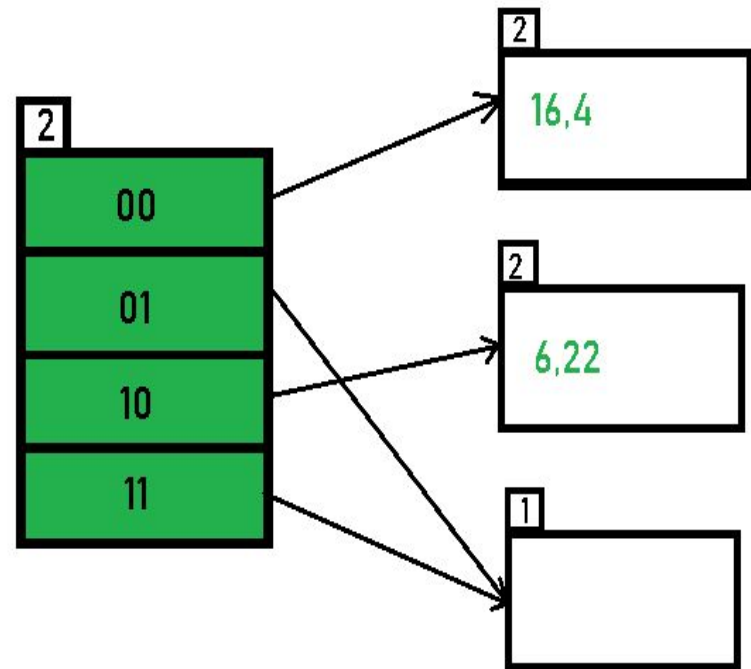
Also, rehashing of numbers present in the overflowing bucket takes place after the split.

And, since the global depth is incremented by 1, now, the global depth is 2.

Hence, 16,4,6,22 are now rehashed w.r.t 2 LSBs.

[16(10000),4(100),6(110),22(10110)]

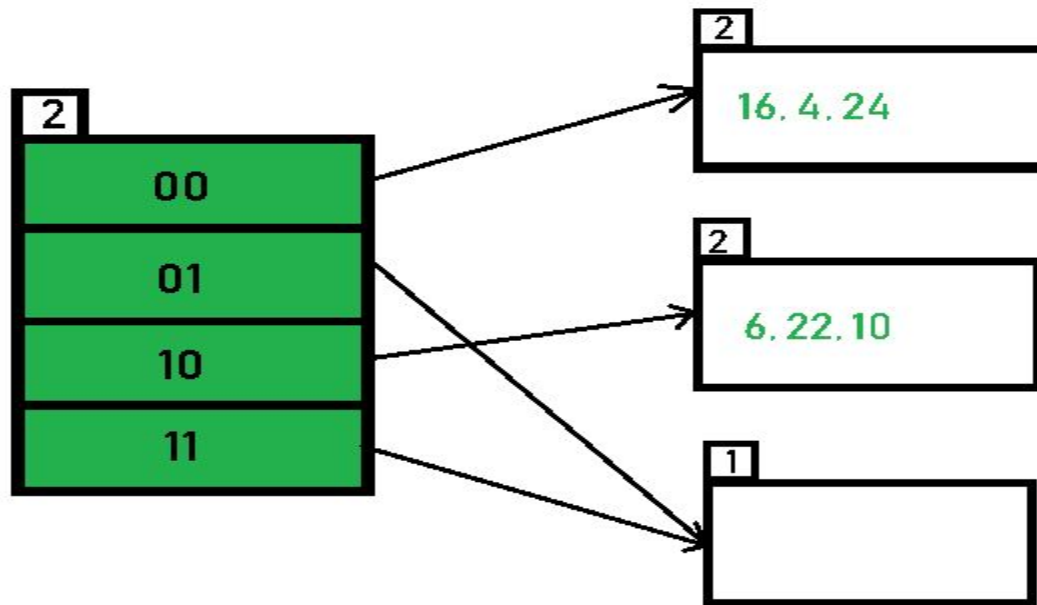
After Bucket Split and Directory Expansion



Note : the bucket which was underflow has remained untouched. But, since the number of directories has doubled, we now have 2 directories 01 and 11 pointing to the same bucket. This is because the local-depth of the bucket has remained 1. And, any bucket having a local depth less than the global depth is pointed-to by more than one directories.

Extendible Hashing

Inserting 24 and 10: 24(11000) and 10 (1010) can be hashed based on directories with id 00 and 10. Here, we encounter no overflow condition

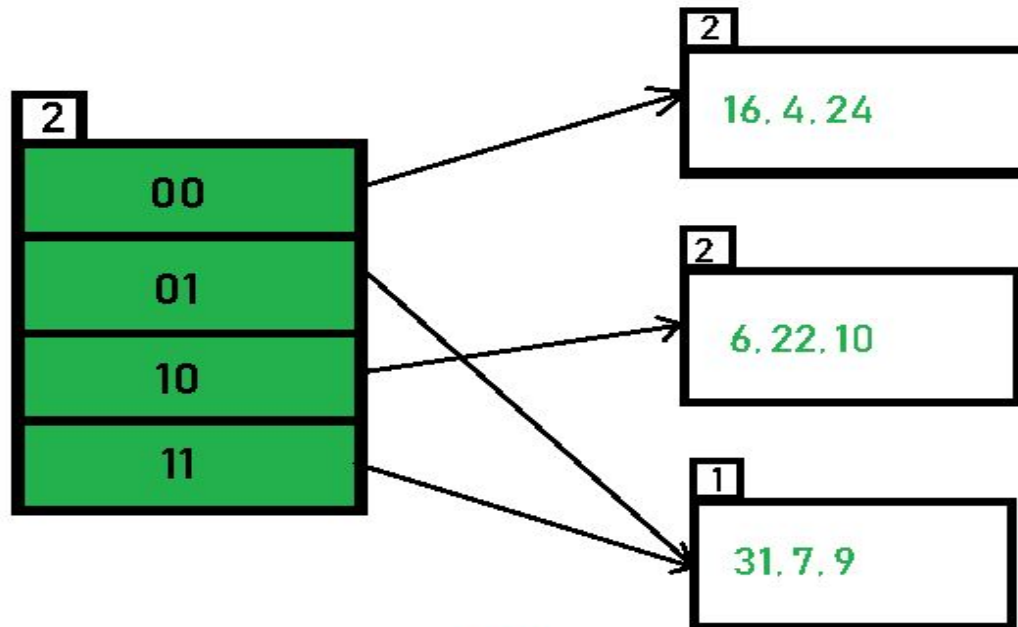


$\text{Hash}(24) = 11000$

$\text{Hash}(10) = 1010$

Extendible Hashing

Inserting 31,7,9: All of these elements[31(11111), 7(111), 9(1001)] have either 01 or 11 in their LSBs. Hence, they are mapped on the bucket pointed out by 01 and 11. We do not encounter any overflow condition here.



$Hash(31) = 11111$

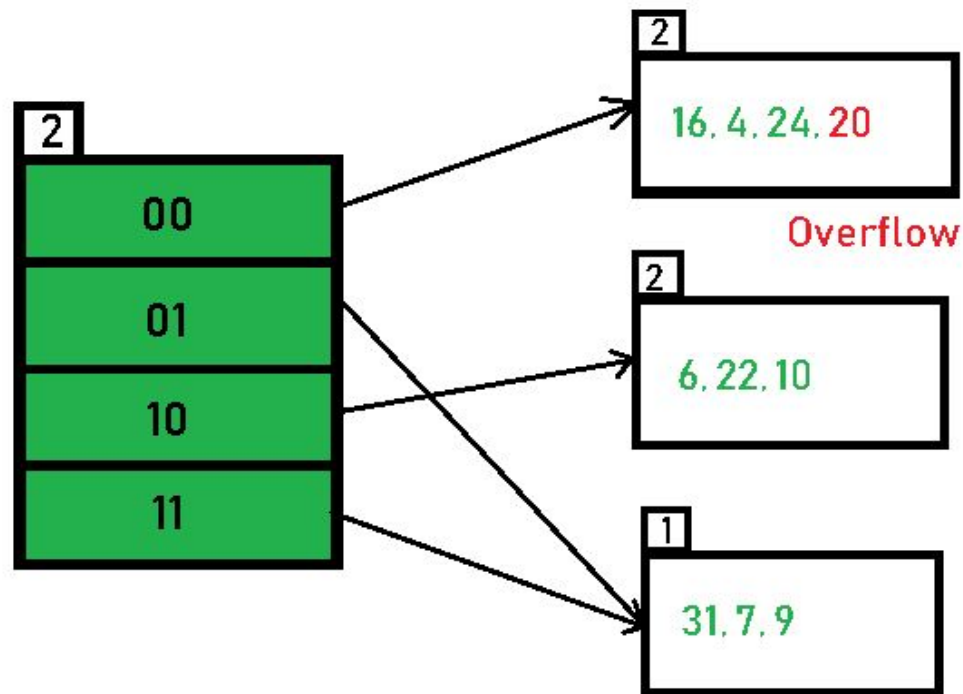
$Hash(7) = 111$

$Hash(9) = 1001$

Extendible Hashing

Inserting 20: Insertion of data element 20 (10100) will again cause the overflow problem

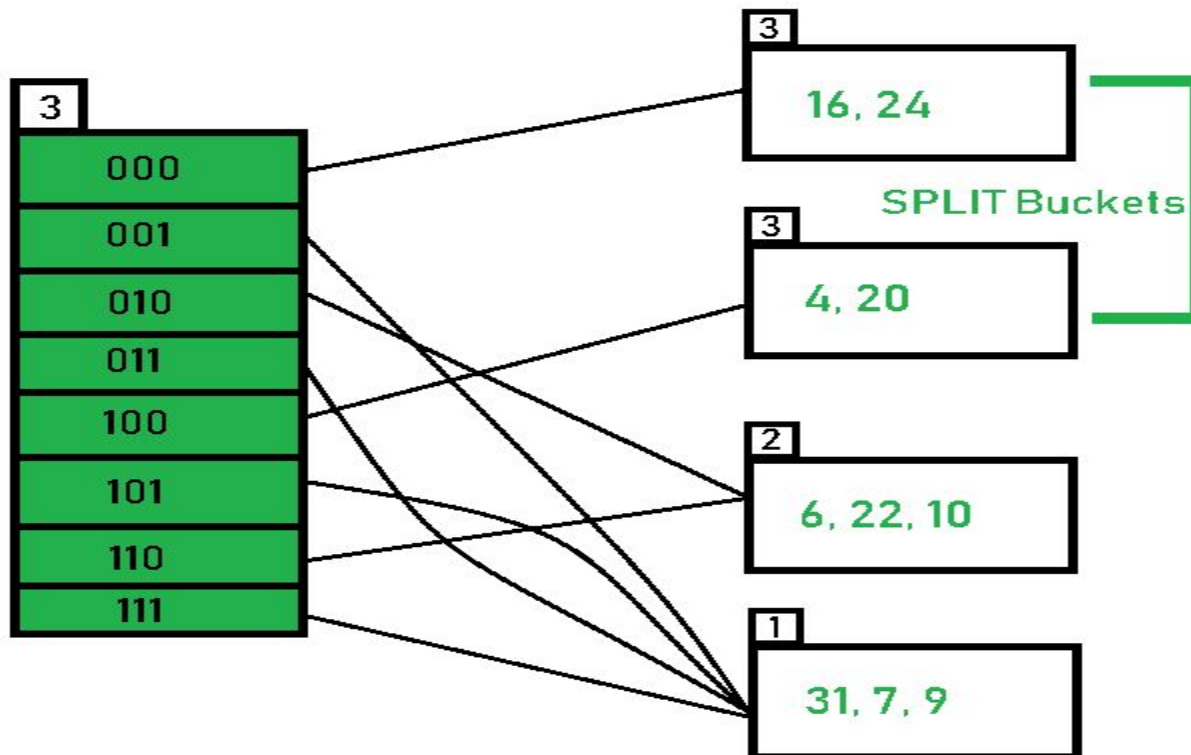
Overflow, Local Depth = Global Depth



$\text{Hash}(20) = 10100$

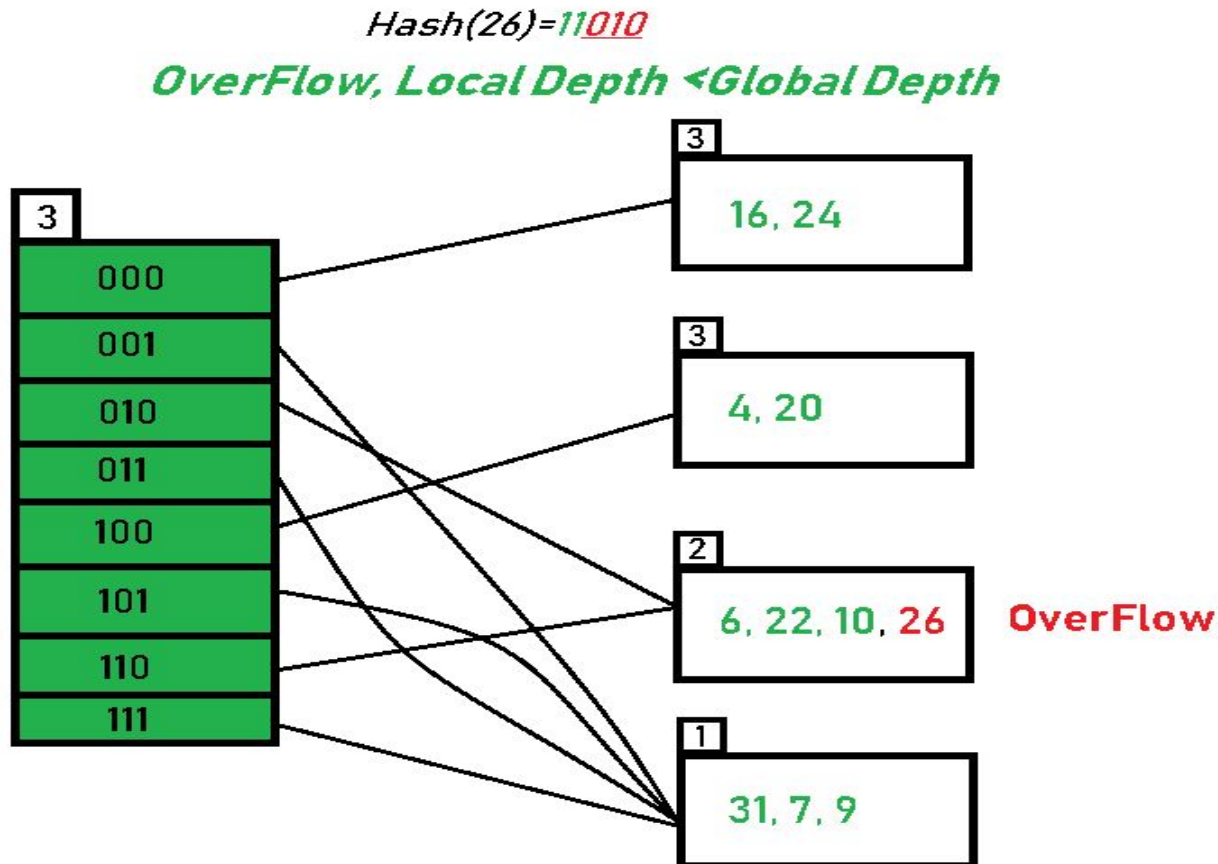
Extendible Hashing

20 is inserted in bucket pointed out by 00. As directed by **Step 7-Case 1**, since the **local depth of the bucket = global-depth**, directory expansion (doubling) takes place along with bucket splitting. Elements present in overflowing bucket are rehashed with the new global depth. Now, the new Hash table looks like this:



Extendible Hashing

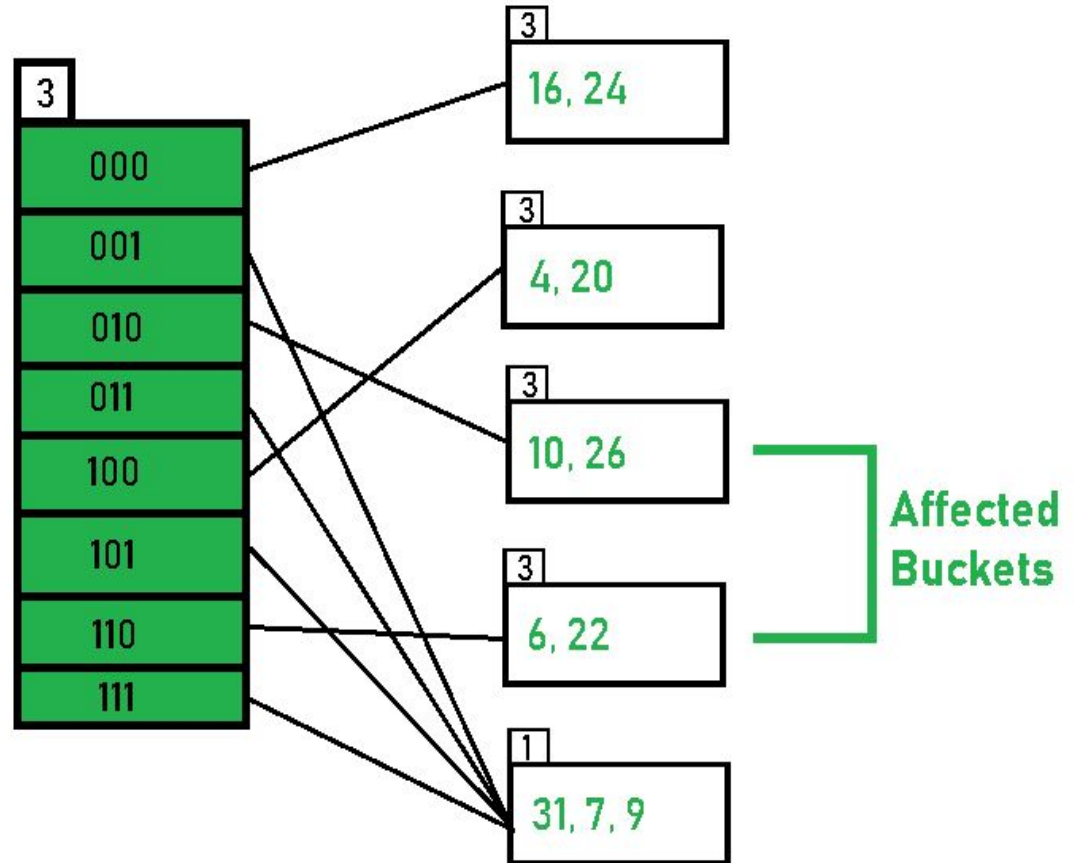
Inserting 26: Global depth is 3. Hence, 3 LSBs of 26(11010) are considered. Therefore 26 best fits in the bucket pointed out by directory 010.



Extendible Hashing

The bucket overflows, and, as directed by **Step 7-Case 2**, since the **local depth of bucket** $< \text{Global depth}$ ($2 < 3$), directories are not doubled but, only the bucket is split and elements are rehashed.

Finally, the output of hashing the given list of numbers is obtained.



Extendible Hashing

Key Observations:

- ❑ A Bucket will have more than one pointers pointing to it if its local depth is less than the global depth.
- ❑ When overflow condition occurs in a bucket, all the entries in the bucket are rehashed with a new local depth.
- ❑ If Local Depth of the overflowing bucket The size of a bucket cannot be changed after the data insertion process begins.

Extendible Hashing

Advantages:

- ❑ Data retrieval is less expensive (in terms of computing).
- ❑ No problem of Data-loss since the storage capacity increases dynamically.
- ❑ With dynamic changes in hashing function, associated old values are rehashed w.r.t the new hash function.

❑ Limitations Of Extendible Hashing:

- ❑ Directory size may increase significantly if several records are hashed on the same directory while keeping the record distribution non-uniform.
- ❑ Size of every bucket is fixed.
- ❑ Memory is wasted in pointers when the global depth and local depth difference becomes drastic.
- ❑ This method is complicated to code.

Applications of Hashing

Hash tables are used in a wide variety of applications, including:

- ❑ **Databases:** Storing and retrieving data based on unique keys
- ❑ **Caching:** Storing frequently accessed data for faster retrieval
- ❑ **Symbol Tables:** Mapping identifiers to their values in programming languages
- ❑ **Network Routing:** Determining the best path for data packets